

# Crop Health Monitoring System using Drone

*A report submitted in partial fulfilment of the requirements  
for the degree of M.Tech. Electronics and Communication Engineering  
with specialization in Drones and IoT*

*by*

S MD PARVEZ  
(Roll No: 224EC2003)

Under the Guidance of

**Dr. K. Krishna Naik**

Associate Professor Dept. of ECE, IIITDM Kurnool



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING  
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,  
DESIGN AND MANUFACTURING, KURNOOL

April 2026

**Title of the Project:** Crop Health Monitoring System using Drone

**Name of the Student:** S Md Parvez

**Examiner(s):**

-----  
-----

**Supervisor(s):**

-----  
-----

**Head of the Department:**

-----

**Date:**

**Place:** IIITDM Kurnool

# Certificate

I, **S MD PARVEZ**, with Roll No: **224EC2003** hereby declare that the material presented in the Project Report titled **Crop Health Monitoring System using Drone** represents original work carried out by me in the **Department of Electronics and Communication Engineering** at the **Indian Institute of Information Technology, Design and Manufacturing, Kurnool** during the years **2024–2026**.

With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property.
- I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date:

Student's Signature

In my capacity as supervisor of the above-mentioned work, I certify that the work presented in this Report is carried out under my supervision, and is worthy of consideration for the requirements of M.Tech. Project work.

Advisor's name:

Advisor's Signature

# *Abstract*

This report presents a drone-based Crop Health Monitoring System developed to address major challenges in modern agriculture, especially in large-scale farming where regular monitoring is difficult. Traditional crop inspection methods rely on manual observation, which is time-consuming, labor-intensive, and often inaccurate. As a result, crop diseases and ineffective pesticide usage may not be identified at the right time, leading to reduced crop yield and productivity.

Existing systems have explored the use of image processing and machine learning for disease detection, but many of them are limited to controlled environments and focus only on identifying diseases. They often fail to provide information about disease severity, track crop health over time, or integrate drone-based data with real-time mobile applications, which limits their practical usage in field conditions.

To overcome these limitations, the proposed system integrates drone imaging, artificial intelligence, and mobile technology into a unified platform. The system supports crops such as Chilli, Coconut, Corn, Cotton, Maize, Onion, Paddy, Sugarcane, Tomato, and Wheat. It analyzes images captured from drones or uploaded by users using deep learning models for crop verification and disease detection. Additionally, an HSV-based image processing method is used to estimate disease severity by calculating affected and healthy areas.

The system also enables monitoring of crop health over time by comparing conditions before and after pesticide spraying and displaying results through simple graphical trends. It is implemented as an Android application using ONNX Runtime, allowing efficient operation without continuous internet connectivity. Experimental results demonstrate good accuracy in disease detection and severity estimation, making the system a practical and user-friendly solution for improving agricultural decision-making and productivity.

## *Acknowledgements*

I would like to express my sincere gratitude to **Dr. K. Krishna Naik**, Associate Professor, Department of Electronics and Communication Engineering, **IITDM Kurnool** for motivating and supervising me through this project. I would like to thank him for his guidance and support in project completion.

I am also grateful to the faculty members of the Electronics and Communication Engineering Department for their support, feedback, and inspiration. Their knowledge and encouragement provided me with a strong academic foundation and motivated me to explore my project with confidence and curiosity.

I owe my heartfelt thanks to my family for their unconditional love, patience, and understanding. Their constant support and belief in my abilities gave me the strength to overcome obstacles and stay motivated, even during the most challenging phases of this project.

I would also like to thank my friends, who have been a source of encouragement, motivation, and constructive discussions. Their support, camaraderie, and positivity made this journey enjoyable and memorable. This project would not have been possible without the collective support, guidance, and encouragement I have received, and I am truly grateful to everyone who contributed to it in some way.

S Md Parvez

224EC2003

# Contents

|                                                                |             |
|----------------------------------------------------------------|-------------|
| <b>Evaluation Sheet</b>                                        | <b>i</b>    |
| <b>Certificate</b>                                             | <b>ii</b>   |
| <b>Abstract</b>                                                | <b>iii</b>  |
| <b>Acknowledgements</b>                                        | <b>iv</b>   |
| <b>Contents</b>                                                | <b>v</b>    |
| <b>List of Figures</b>                                         | <b>viii</b> |
| <b>Abbreviations</b>                                           | <b>x</b>    |
| <br>                                                           |             |
| <b>1 Introduction</b>                                          | <b>1</b>    |
| 1.1 Features of Crop Health Monitoring System . . . . .        | 2           |
| 1.2 Motivation . . . . .                                       | 3           |
| 1.3 Objective . . . . .                                        | 4           |
| 1.4 Problem Statements . . . . .                               | 4           |
| 1.5 Organization of the Report . . . . .                       | 4           |
| <br>                                                           |             |
| <b>2 Literature Review</b>                                     | <b>6</b>    |
| 2.1 Introduction . . . . .                                     | 6           |
| 2.2 UAV-Based Agricultural Monitoring . . . . .                | 6           |
| 2.3 Deep Learning for Crop Disease Detection . . . . .         | 7           |
| 2.4 Role of Datasets in Agricultural AI . . . . .              | 7           |
| 2.5 Image Processing for Disease Severity Estimation . . . . . | 8           |
| 2.6 Mobile and Edge-Based AI Systems . . . . .                 | 8           |
| 2.7 Crop Verification and Model Reliability . . . . .          | 9           |
| 2.8 Limitations of Existing Systems . . . . .                  | 9           |
| 2.9 Research Gap . . . . .                                     | 9           |

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| 2.10 Conclusion . . . . .                                                        | 10        |
| <b>3 Design of the Crop Health Monitoring System</b>                             | <b>11</b> |
| 3.1 System Architecture Design . . . . .                                         | 11        |
| 3.2 Image Acquisition and ROI Selection . . . . .                                | 12        |
| 3.3 Crop Verification and Disease Detection . . . . .                            | 13        |
| 3.4 Severity Estimation using RGB to HSV Conversion . . . . .                    | 13        |
| 3.5 Database Integration and Result Storage . . . . .                            | 15        |
| 3.6 Graph Visualization and Output Analysis . . . . .                            | 15        |
| 3.7 Recovery Reset Mechanism . . . . .                                           | 16        |
| <b>4 Development of Training Models of Crops and Crop Verifier</b>               | <b>17</b> |
| 4.1 Deep Learning Architecture for Crop and Disease Classification . . . . .     | 17        |
| 4.1.1 Input Layer . . . . .                                                      | 18        |
| 4.1.2 Stem Layer (Initial Feature Extraction) . . . . .                          | 19        |
| 4.1.3 Dense Block 1 (6 Layers) . . . . .                                         | 19        |
| 4.1.4 Transition Layer 1 . . . . .                                               | 19        |
| 4.1.5 Dense Block 2 (12 Layers) . . . . .                                        | 20        |
| 4.1.6 Transition Layer 2 . . . . .                                               | 20        |
| 4.1.7 Dense Block 3 (48 Layers — The Deepest Block) . . . . .                    | 20        |
| 4.1.8 Transition Layer 3 . . . . .                                               | 20        |
| 4.1.9 Dense Block 4 (32 Layers — Final High-Level Feature Extraction) . . . . .  | 20        |
| 4.1.10 Global Average Pooling (Spatial Feature Compression) . . . . .            | 21        |
| 4.1.11 Classifier Head (Final Decision Maker) . . . . .                          | 21        |
| 4.1.12 Final Output — Crop Verification / Disease Classification . . . . .       | 21        |
| 4.2 Dataset Implementation and Preparation . . . . .                             | 22        |
| 4.2.1 Dataset Collection and Sources . . . . .                                   | 22        |
| 4.2.2 Dataset Organization and Structure . . . . .                               | 22        |
| 4.2.3 Data Processing and Augmentation . . . . .                                 | 22        |
| 4.2.4 Selected Crops and Image Distribution . . . . .                            | 23        |
| 4.3 Training Workflow of the Model . . . . .                                     | 24        |
| 4.4 Crop Verification Model . . . . .                                            | 25        |
| 4.5 Disease Detection Model . . . . .                                            | 25        |
| 4.6 ONNX Model Export and Deployment . . . . .                                   | 26        |
| <b>5 Development of Crop Health Monitoring System Android Application</b>        | <b>27</b> |
| 5.1 Development Environment and Technologies Used . . . . .                      | 27        |
| 5.2 User Interface Module Design . . . . .                                       | 28        |
| 5.3 Image Upload and Processing Module . . . . .                                 | 30        |
| 5.4 Crop Verification and Disease Detection Trained Models Integration . . . . . | 31        |
| 5.5 Severity Estimation and Result Display . . . . .                             | 31        |
| 5.6 Database Implementation using Room . . . . .                                 | 33        |
| 5.7 Graph Visualization Module . . . . .                                         | 33        |
| 5.8 Background Processing and Performance Optimization . . . . .                 | 34        |

---

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>6</b> | <b>Results and Discussions</b>                                  | <b>35</b> |
| 6.1      | Model Performance Evaluation . . . . .                          | 36        |
| 6.2      | Confusion Matrix Analysis . . . . .                             | 61        |
| 6.3      | Model Accuracy Evaluation . . . . .                             | 64        |
| <b>7</b> | <b>Conclusion and Future Work</b>                               | <b>66</b> |
| 7.1      | Conclusion . . . . .                                            | 66        |
| 7.2      | Future Work . . . . .                                           | 67        |
| <b>A</b> | <b>Application Code</b>                                         | <b>69</b> |
| A.1      | Installation Libraries Requirements : . . . . .                 | 69        |
| A.2      | Implementation of the Crop Health Monitoring System . . . . .   | 70        |
| A.2.1    | Crop Verifier Model Training . . . . .                          | 70        |
| A.2.2    | Individual Crop Disease Classification Model Training . . . . . | 74        |
| A.2.3    | Confusion Matrix and Classification Report Generation . . . . . | 78        |
|          | <b>Bibliography</b>                                             | <b>81</b> |

# List of Figures

|      |                                                                            |    |
|------|----------------------------------------------------------------------------|----|
| 3.1  | Overall System Architecture of the Proposed Crop Health Monitoring System  | 11 |
| 3.2  | HSV Color Space Based Severity Estimation . . . . .                        | 13 |
| 3.3  | Disease Progress Trend Over Days . . . . .                                 | 15 |
| 4.1  | DenseNet-201 Architecture . . . . .                                        | 18 |
| 4.2  | Representative Dataset for Each Crop and Disease Classes . . . . .         | 23 |
| 4.3  | Workflow of the Model Training . . . . .                                   | 24 |
| 5.1  | Main User Interface of Crop Health Monitoring System Application . . . . . | 29 |
| 5.2  | Image Upload and Preview Interface . . . . .                               | 30 |
| 5.3  | Disease Detection Output Display . . . . .                                 | 32 |
| 5.4  | Crop Health Database . . . . .                                             | 33 |
| 5.5  | Crop Health Trend Graph Visualization . . . . .                            | 34 |
| 6.1  | Chilli Crop Disease Detected . . . . .                                     | 37 |
| 6.2  | Coconut Crop Disease Detected . . . . .                                    | 38 |
| 6.3  | Corn Crop Disease Detected . . . . .                                       | 38 |
| 6.4  | Cotton Crop Disease Detected . . . . .                                     | 39 |
| 6.5  | Onion Crop Disease Detected . . . . .                                      | 39 |
| 6.6  | Paddy Crop Disease Detected . . . . .                                      | 39 |
| 6.7  | Sugarcane Crop Disease Detected . . . . .                                  | 40 |
| 6.8  | Tomatoes Crop Disease Detected . . . . .                                   | 40 |
| 6.9  | Wheat Crop Disease Detected . . . . .                                      | 40 |
| 6.10 | Crop Verification Failed (Incorrect Crop Uploaded) . . . . .               | 41 |
| 6.11 | Before Spray Disease Detection . . . . .                                   | 41 |
| 6.12 | After Spray Disease Detection . . . . .                                    | 42 |
| 6.13 | After Spray Disease Detection . . . . .                                    | 43 |
| 6.14 | Disease Progress Analysis . . . . .                                        | 44 |
| 6.15 | Before and After Spray Disease Detection . . . . .                         | 45 |
| 6.16 | Disease Progress Analysis . . . . .                                        | 46 |
| 6.19 | Before and After Spray Disease Detection . . . . .                         | 46 |
| 6.20 | Disease Progress Analysis . . . . .                                        | 47 |
| 6.23 | Before and After Spray Disease Detection . . . . .                         | 47 |
| 6.26 | Before and After Spray Disease Detection . . . . .                         | 48 |
| 6.27 | Disease Progress Analysis . . . . .                                        | 48 |
| 6.28 | Before and After Spray Disease Detection . . . . .                         | 49 |

|                                                          |    |
|----------------------------------------------------------|----|
| 6.29 After Spray Disease Detection . . . . .             | 50 |
| 6.30 Disease Progress Analysis . . . . .                 | 50 |
| 6.31 Before and After Spray Disease Detection . . . . .  | 51 |
| 6.32 Disease Progress Analysis . . . . .                 | 52 |
| 6.33 Before and After Spray Disease Detection . . . . .  | 52 |
| 6.34 After Spray Disease Detection . . . . .             | 54 |
| 6.35 Disease Progress Analysis . . . . .                 | 54 |
| 6.36 Before and After Spray Disease Detection . . . . .  | 56 |
| 6.37 After Spray Disease Detection . . . . .             | 57 |
| 6.38 Disease Progress Analysis . . . . .                 | 57 |
| 6.39 Before and After Spray Disease Detection . . . . .  | 59 |
| 6.40 Disease Progress Analysis . . . . .                 | 59 |
| 6.41 Before and After Spray Disease Detection . . . . .  | 60 |
| 6.42 After Spray Disease Detection . . . . .             | 60 |
| 6.43 Disease Progress Analysis . . . . .                 | 61 |
| 6.44 Chilli Crop . . . . .                               | 62 |
| 6.45 Coconut Crop . . . . .                              | 62 |
| 6.46 Corn Crop . . . . .                                 | 62 |
| 6.47 Cotton Crop . . . . .                               | 62 |
| 6.48 Maize Crop . . . . .                                | 63 |
| 6.49 Onion Crop . . . . .                                | 63 |
| 6.50 Paddy Crop . . . . .                                | 63 |
| 6.51 Sugarcane Crop . . . . .                            | 63 |
| 6.52 Tomatos Crop . . . . .                              | 63 |
| 6.53 Wheat Crop . . . . .                                | 63 |
| 6.54 Crop Verifier . . . . .                             | 64 |
| 6.55 Accuracy of Crop Disease Detection Models . . . . . | 65 |

# Abbreviations

|             |                                                                    |
|-------------|--------------------------------------------------------------------|
| <b>AI</b>   | <b>A</b> rtificial <b>I</b> ntelligence                            |
| <b>ML</b>   | <b>M</b> achine <b>L</b> earning                                   |
| <b>DL</b>   | <b>D</b> ee <b>p</b> <b>L</b> earning                              |
| <b>CNN</b>  | <b>C</b> onvolution <b>N</b> eural <b>N</b> etwork                 |
| <b>TNAU</b> | <b>T</b> amil <b>N</b> adu <b>A</b> gricultural <b>U</b> niversity |
| <b>ONNX</b> | <b>O</b> pen <b>N</b> eural <b>N</b> etwork <b>E</b> xchange       |

*Dedicated to Farmers*

# Chapter 1

## Introduction

In recent years, technology has started playing an important role in improving agriculture. The use of Artificial Intelligence (AI) and drones has made it easier to monitor crops and detect diseases at an early stage. In this project, we have developed a Crop Health Monitoring System using Drone, which helps in analyzing crop conditions in a simple and efficient way. The system uses image processing and machine learning techniques to study crop images and provide real-time information about their health.

One of the key features of this system is its ability to automatically analyze crop images and give useful details such as the type of disease, the affected area, and the overall health of the crop. The user can either capture an image or upload it into the system, and the system will process it to detect diseases and calculate the percentage of affected and healthy areas. It also allows comparison of crop conditions before and after spraying pesticides, which helps in understanding whether the treatment is effective or not. This makes it easier for farmers to take better decisions based on accurate information.

The main aim of this project is to reduce the difficulties faced in traditional farming methods, where crop monitoring is done manually. Manual inspection is time-consuming and may not always be accurate, especially for large fields. By using AI and drone-based technology, the system helps in early disease detection, reduces manual effort, and supports monitoring of large agricultural areas. It also contributes to modern farming techniques by promoting data-driven decisions.

In this report, we explain the design and working of the Crop Health Monitoring System. The report covers the system architecture, methods used for disease detection, and techniques such as HSV-based severity estimation. It also describes how machine

learning models are used and how the system is implemented as a mobile application. Finally, the results and graphs generated by the system are presented to show its effectiveness in monitoring crop health.

## 1.1 Features of Crop Health Monitoring System

AI-based Crop Health Monitoring Systems include several useful features that make them helpful for farmers and agricultural researchers. These features are designed to make crop monitoring easier, faster, and more accurate by using modern technology.

- **Automated Crop Detection:** The system can automatically identify the type of crop from the uploaded image. This means the user does not need to select the crop manually, which helps reduce mistakes and makes the process easier.
- **Disease Detection:** The system uses machine learning techniques to detect diseases in crops. It can recognize different types of diseases and also determine whether the crop is healthy or affected.
- **Severity Estimation:** The system calculates how much of the crop is affected by the disease. It uses image processing to find the percentage of diseased and healthy areas, giving a clear idea about the crop condition.
- **Spray Effectiveness Monitoring:** The system compares crop conditions before and after spraying pesticides. This helps in understanding whether the treatment is working and if the disease is reducing.
- **Graph-based Trend Analysis:** The system shows crop health changes over time using graphs. This makes it easier to track whether the crop condition is improving or getting worse over different days.
- **Recovery Detection and Reset Mechanism:** When the crop becomes healthy, the system automatically resets the monitoring cycle. This allows the system to start fresh tracking for future observations.
- **User-Friendly Interface:** The system is designed to be simple and easy to use. Even users with less technical knowledge can upload images and understand the results without difficulty.
- **Real-time Analysis:** The system processes images quickly and provides results instantly. This helps farmers take timely decisions to manage their crops effectively.

## 1.2 Motivation

The idea for developing the Crop Health Monitoring System using Drone comes from the need to make crop monitoring easier, faster, and more accurate. In traditional farming, farmers usually check crops manually, which takes a lot of time and effort. It is also difficult to monitor large fields regularly, and sometimes diseases are detected too late. Because of this, there is a need for a smart system that can help farmers monitor their crops more efficiently.

Here are some of the key benefits:

- **Early Detection of Crop Diseases:** The system helps in identifying diseases at an early stage. This allows farmers to take quick action and prevent the disease from spreading to other plants.
- **Better Decision Making:** By providing clear information about crop health, the system helps farmers decide when to use pesticides and how to manage their crops effectively.
- **Reduced Manual Work:** Instead of checking crops manually every day, the system automatically analyzes images and gives results. This saves time and effort for farmers.
- **Efficient Use of Pesticides:** The system compares crop conditions before and after spraying. This helps farmers understand whether the treatment is working and avoids unnecessary use of chemicals.
- **Support for Modern Farming:** The system uses advanced technology like AI and drone imaging, which supports modern and smart farming methods.
- **Monitoring Large Fields Easily:** With the help of drones, large agricultural areas can be monitored without much difficulty, which is not possible with manual inspection.
- **Continuous Crop Monitoring:** The system allows farmers to track crop health regularly over time, helping them understand how the crop is improving or getting affected.

### 1.3 Objective

The main objective of the Crop Health Monitoring System using Drone is to make crop monitoring easier and more accurate by using artificial intelligence and image processing. The system is designed to help identify crop diseases, estimate how much of the crop is affected, and track the overall health of crops over time. It helps farmers understand the condition of their crops and take the right actions at the right time.

Another important objective is to develop a simple and user-friendly mobile application where users can upload crop images and get results instantly. The system provides clear information about crop health and also shows how it changes over time using graphs. Overall, the goal of this project is to reduce manual work, improve decision-making, and support modern farming by providing an easy and efficient solution for crop monitoring.

### 1.4 Problem Statements

- Develop a system that can automatically detect crop diseases by using artificial intelligence on images captured by drones or uploaded by users.
- Create a solution that can identify different crop types and detect the diseases affecting them accurately using machine learning techniques.
- Design a method to estimate how much of the crop is affected by the disease by calculating the percentage of affected and healthy areas using image processing.
- Solve the difficulty of checking whether pesticide spraying is effective by comparing crop conditions before and after spraying.
- Handle the need for continuous monitoring of crop health by storing data and showing changes over time using graphs.
- Develop an easy-to-use mobile application that allows farmers to upload images and understand the results without any difficulty.

### 1.5 Organization of the Report

The project report is divided into seven chapters to clearly explain the complete development and working of the Crop Health Monitoring System.

- This Chapter 1 explains the introduction, motivation, objectives, and problem statements of the project. It also discusses the importance of using Artificial Intelligence and drone technology in agriculture and highlights the key features of the system.
- Chapter 2 presents the literature survey, where existing methods and technologies related to crop disease detection and monitoring are discussed. It also includes the tools, software, and techniques used in this project.
- Chapter 3 describes the system design and overall architecture of the proposed model. It explains how different components of the system work together.
- Chapter 4 focuses on the implementation of the system, including image processing, crop verification, disease detection, and severity estimation methods.
- Chapter 5 explains the development of the mobile application and how the system is integrated into a user-friendly interface.
- Chapter 6 presents the results obtained from the system, including disease detection outputs, severity estimation, graphical analysis of crop health and confusion Matrix Analysis.
- Chapter 7 provides the conclusion of the project and discusses future scope. It includes possible improvements and additional features that can enhance the system further.

## Chapter 2

# Literature Review

### 2.1 Introduction

Recently, the agricultural industry has seen significant evolution with the introduction of advanced technologies like Artificial Intelligence (AI), image processing, and Unmanned Aerial Vehicles (UAVs). Traditional crop monitoring systems depend extensively on human intervention and are time-consuming and error-prone. Early detection of crop diseases and estimation of crop health are important aspects of agriculture, as they have direct impacts on the efficiency and yield of farms.

Many researchers have recently studied automated crop monitoring techniques that leverage drones for image collection and AI models for analysis. This chapter contains the detailed study of various works related to UAV-based agricultural monitoring, application of deep learning in disease detection, dataset usage in the field, image processing for severity estimation, and deployment systems on mobile devices.

### 2.2 UAV-Based Agricultural Monitoring

The use of UAVs in agriculture has been increasing due to the benefits they offer in comparison to satellite systems. UAVs have much better spatial resolution than satellites due to which they are more beneficial for crop monitoring.

Using image processing algorithms, Sharada et al. [7] have shown that drones are useful for crop health monitoring and stress detection. Similarly, Pham et al. [8] have used UAV

imagery to detect diseases in rice plants with the help of deep learning models, proving that aerial imagery has high potential in agriculture.

Both studies validate that the application of UAVs in agriculture is a very effective technique.

## 2.3 Deep Learning for Crop Disease Detection

Recent advancements in AI have led to the creation of highly sophisticated machine learning models, including deep neural networks. Deep learning has enabled researchers to develop highly precise algorithms that classify diseases in crops.

In a pioneering study in the domain, Mohanty et al. [1] have developed a deep learning model for crop disease classification with accuracy higher than 99% using the PlantVillage dataset. Similarly, in another study, Sladojevic et al. [4] have validated that deep learning models can accurately detect diseases with the help of leaf images.

Ferentinos [6] compared various deep learning architectures and concluded that Convolutional Neural Network (CNN)-based models are superior to traditional classifiers. In a similar study, Singh et al. [3] emphasize the efficiency of convolutional neural networks in classifying crop diseases.

Moreover, Too et al. [5] show that fine-tuning of the models leads to better performance and significantly reduces the amount of training data needed. In a specific domain-focused study, Zhang et al. [14] have shown that CNNs can be used to classify diseases in maize plants effectively.

All above-discussed studies show that deep learning has proven its usefulness in automated detection of crop diseases.

## 2.4 Role of Datasets in Agricultural AI

As we know that performance of any model highly depends on its dataset. Therefore, availability of appropriate datasets becomes essential for developing effective AI models.

Barbedo [9] shows that both the size and variety of datasets contribute significantly to improving performance and generalizing the model. Similarly, in another study,

Ramcharan et al. [10] have discussed how real-world data helps in training deep learning models.

Many researchers use datasets like PlantVillage, which consists of artificially acquired images that are taken in controlled environments. It decreases their practical value because they fail to take real-world situations into account.

Field-based datasets consisting of images collected from agricultural universities and UAVs are used by many researchers to improve model performance.

## 2.5 Image Processing for Disease Severity Estimation

Although deep learning models perform classification tasks, traditional image processing techniques are used for severity estimation.

Machine learning models and image processing algorithms are used for plant disease detection and identification by Ramesh et al. [12]. In a similar work, Brahimi et al. [11] show that visualization of disease symptoms helps researchers understand the disease better.

Techniques based on HSV color space are often used for segmenting unhealthy portions from healthy portions in images. Such techniques facilitate disease severity estimation.

## 2.6 Mobile and Edge-Based AI Systems

In recent times, there has been an increased need for developing AI models for use on mobile and edge devices.

Foundational knowledge for creating efficient deep learning models was given by LeCun et al. [13]. Recent developments in deep learning architectures and optimization methods have facilitated the development of AI models that can be run on mobile devices.

Many tools and libraries exist that allow deployment of such models on phones. For example, ONNX Runtime and TensorFlow Lite libraries allow models to run on mobiles without requiring access to cloud services.

Mobile-based systems make AI easily accessible for farmers who lack internet connectivity.

## 2.7 Crop Verification and Model Reliability

The most challenging task in developing crop disease detection systems is to ensure that image submitted to the system belongs to the right crop type. Otherwise, the system might detect wrong diseases that will reduce its reliability.

Preprocessing of images and building crop-specific models as suggested by Sladojevic et al. [4] can solve this problem, but researchers are required to develop a separate model for each crop type.

Therefore, recent systems include an additional crop verification step that determines the type of crop and then disease is detected.

## 2.8 Limitations of Existing Systems

Even after making huge advancements in the domain, current systems still face some problems:

- Lack of provision for disease severity estimation
- Limited implementation of drone-imaging technology
- No temporal analysis capabilities
- Absence of spray effectiveness estimation
- No automatic recovery tracking mechanism

Furthermore, many of the models are trained on artificial datasets, so they might not perform reliably in real-world applications.

## 2.9 Research Gap

From literature review, it can be concluded that the following research gaps exist:

- Need for UAV-imaged and AI-based integrated systems
- Continuous crop health monitoring over time

- Lack of evaluation of spraying effectiveness
- Implementation of recovery tracking
- Lack of crop verification mechanisms

## 2.10 Conclusion

From the literature review, it is clear that UAV-based imaging and deep learning techniques have greatly evolved the world of crop monitoring. With the help of deep learning algorithms, accurate disease classification is performed. On the other hand, image processing techniques enable severity estimation. Moreover, mobile-based AI systems increase practicality.

Existing crop monitoring systems have many shortcomings including lack of integrated functionality and poor usability. To cater to the limitations of previous work, the proposed Crop Health Monitoring System integrates all the required features to serve farmers effectively.

## Chapter 3

# Design of the Crop Health Monitoring System

### 3.1 System Architecture Design

The proposed Crop Health Monitoring System is designed as an integrated intelligent framework that combines drone-based image acquisition, artificial intelligence, image processing, and mobile application-based visualization into a single unified platform. The overall design focuses on providing an efficient, real-time, and user-friendly solution for detecting crop diseases and monitoring plant health over time.

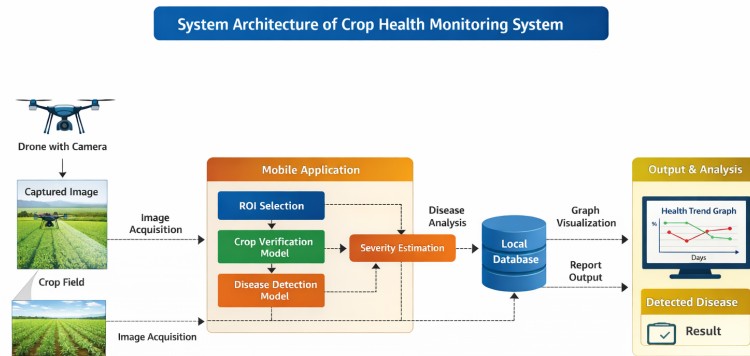


FIGURE 3.1: Overall System Architecture of the Proposed Crop Health Monitoring System

As illustrated in the system architecture diagram provided, the workflow begins with capturing images of crops using a drone equipped with a camera or through direct mobile input. These images are then passed to the mobile application, where multiple processing stages are executed in a sequential manner. The system ensures that each stage contributes to refining the analysis and improving the accuracy of the final output.

The architecture follows a structured pipeline consisting of image acquisition, ROI selection, crop verification, disease detection, severity estimation, database storage, and graphical visualization. This modular design ensures flexibility, scalability, and efficient processing while maintaining simplicity in implementation.

In addition to the system workflow, the way images are captured also has a direct impact on how well the system performs. The drone needs to fly at an appropriate height so that the images are clear enough to show the details of the crop. If the drone flies too high, the images may lose clarity, making it difficult to notice disease symptoms properly. At the same time, flying too close to the crops is not ideal, as it reduces the area covered and makes the process less efficient. So, selecting a suitable flying height is important to get a good balance between image clarity and field coverage. Capturing images in this way helps the system analyze crop conditions more accurately and improves the reliability of disease detection and severity estimation.

## 3.2 Image Acquisition and ROI Selection

The first stage of the system involves capturing high-resolution images of crops either through drone-based imaging or mobile camera input. Drone-based acquisition plays a crucial role in covering large agricultural areas efficiently, while mobile input allows localized analysis.

Once the image is captured, the system performs Region of Interest (ROI) selection. This step is important because raw images often contain background elements such as soil, sky, or surrounding plants that are not relevant for analysis. By selecting only the specific crop region, the system ensures that further processing is focused and accurate.

The ROI selection improves model performance by reducing noise and ensuring that only meaningful visual information is passed to the next stage.

### 3.3 Crop Verification and Disease Detection

After preprocessing, the system performs crop verification using a deep learning model. This model automatically identifies the crop type from the uploaded image by analyzing features such as leaf structure, color patterns, and texture. This eliminates the need for manual crop selection and reduces user errors.

Once the crop is verified, the disease detection model processes the image to identify the type of disease affecting the crop. The model is trained on a dataset containing both healthy and diseased samples, enabling it to classify crop conditions accurately.

If no disease is detected, the system classifies the crop as healthy. Otherwise, it outputs the detected disease along with relevant information. This automated detection significantly improves the speed and reliability of diagnosis compared to traditional manual methods.

### 3.4 Severity Estimation using RGB to HSV Conversion

After identifying the disease, the system estimates the severity of infection using image processing techniques based on HSV color space. This step is essential because it provides a quantitative measure of how much of the crop is affected.

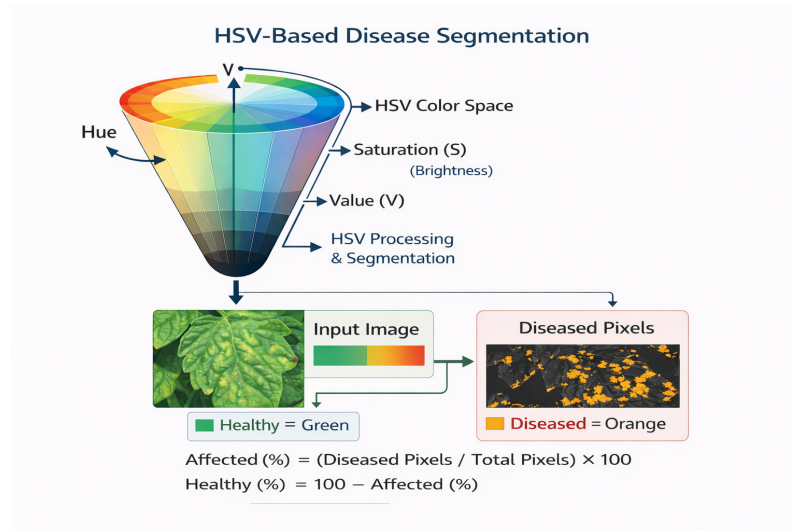


FIGURE 3.2: HSV Color Space Based Severity Estimation

Initially, the input image is in RGB (Red, Green, Blue) format. However, RGB is not suitable for detecting variations in color intensity and brightness effectively. Therefore, the image is converted into HSV (Hue, Saturation, Value) color space.

The conversion from RGB to HSV is performed using the following mathematical expressions:

Let

$$R, G, B \in [0, 1]$$

Then,

$$V = \max(R, G, B)$$

$$S = \frac{V - \min(R, G, B)}{V}$$

Hue ( $H$ ) is calculated as:

$$H = \begin{cases} 60 \times \left( \frac{G-B}{V-\min(R,G,B)} \right), & \text{if } V = R \\ 60 \times \left( 2 + \frac{B-R}{V-\min(R,G,B)} \right), & \text{if } V = G \\ 60 \times \left( 4 + \frac{R-G}{V-\min(R,G,B)} \right), & \text{if } V = B \end{cases}$$

The HSV representation makes it easier to identify diseased regions, as infected areas typically show changes in saturation and brightness.

In this system, pixels are classified as diseased based on threshold conditions applied to saturation and value. The number of diseased pixels is then calculated.

The severity is computed using:

$$\text{Affected Percentage} = (\text{Diseased Pixels} / \text{Total Pixels}) \times 100$$

$$\text{Healthy Percentage} = 100 - \text{Affected Percentage}$$

This mathematical model transforms visual information into numerical values, enabling precise monitoring of crop health.

### 3.5 Database Integration and Result Storage

After severity estimation, the results are stored in a local database within the mobile application. The system uses Room Database to manage data efficiently.

Each record includes information such as crop type, detected disease, severity percentage, spray stage, and timestamp. This stored data enables the system to maintain a history of observations and supports further analysis.

The database also plays a key role in tracking crop health over multiple days. When the crop becomes fully healthy, the system automatically resets the stored data to begin a new monitoring cycle.

### 3.6 Graph Visualization and Output Analysis

The final stage of the system involves presenting the results to the user through graphical visualization and output analysis. The system generates a line graph that represents crop health trends over time.

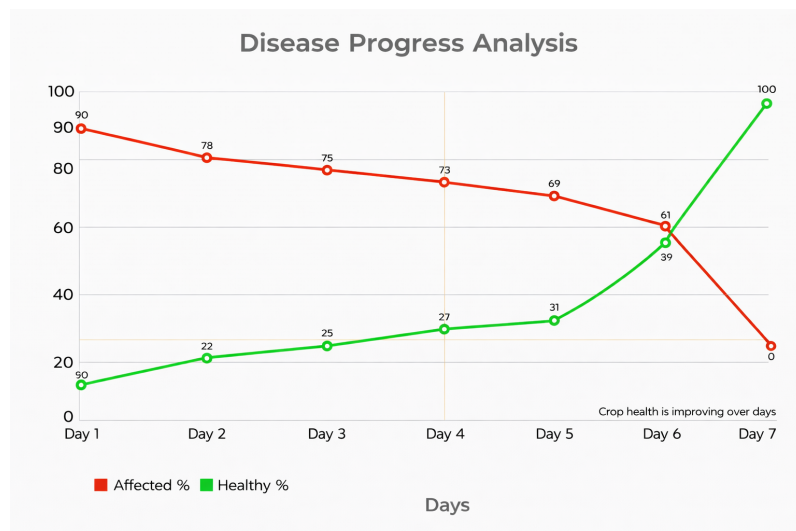


FIGURE 3.3: Disease Progress Trend Over Days

The graph includes:

- X-axis representing days (Day 1, Day 2, etc.)
- Y-axis representing percentage values (affected and healthy)

This visualization helps users understand whether the crop condition is improving, worsening, or stable. Additionally, the system displays the detected disease and provides a clear result summary.

This stage enhances decision-making by converting complex data into an easily understandable format.

### **3.7 Recovery Reset Mechanism**

The system includes a recovery mechanism to restart monitoring when the crop becomes healthy. If the disease is detected as healthy and the affected area is less than or equal to 5%, previous records are cleared.

A new monitoring cycle starts from Day 1, ensuring accurate tracking of future disease progression.

## Chapter 4

# Development of Training Models of Crops and Crop Verifier

The development of accurate and efficient machine learning models is a crucial component of the Crop Health Monitoring System using Drone. The performance of the system largely depends on the ability of these models to correctly identify crop types and detect diseases under varying environmental conditions. This chapter focuses on the design, training, and deployment of deep learning models used for crop verification and disease classification.

The models are developed using advanced convolutional neural network architectures and trained on carefully prepared datasets. The overall process includes dataset preparation, preprocessing, model training, validation, and export into a deployable format suitable for mobile applications.

### 4.1 Deep Learning Architecture for Crop and Disease Classification

The system utilizes a deep learning architecture based on Dense Convolutional Networks (DenseNet), which are known for their efficient feature reuse and improved gradient flow. The architecture consists of multiple dense blocks, transition layers, and a classification head.

In this model, each dense block contains several convolutional layers where each layer receives inputs from all previous layers. This dense connectivity helps in capturing both low-level and high-level features effectively.

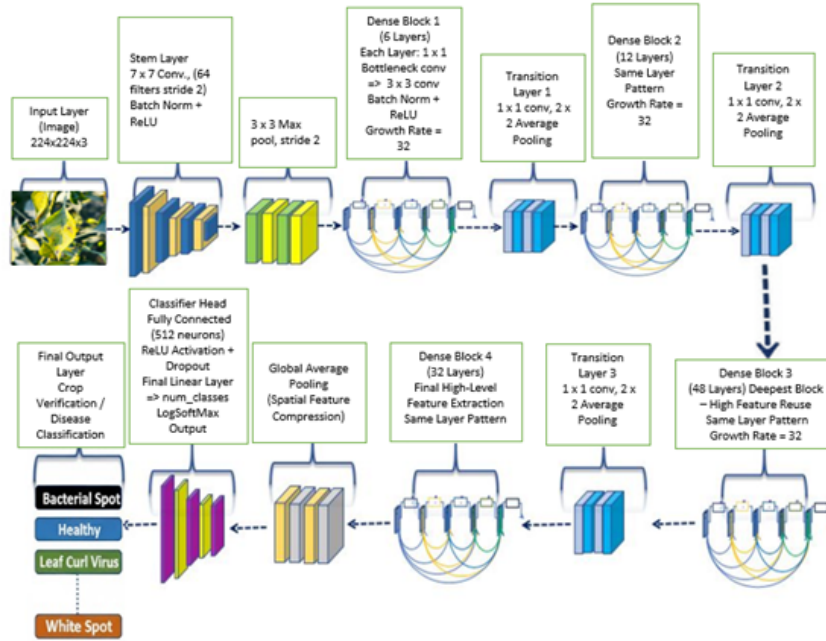


FIGURE 4.1: DenseNet-201 Architecture

The architecture includes the following stages:

- Initial convolution layer for feature extraction
- Dense blocks for deep feature learning
- Transition layers for dimensionality reduction
- Global average pooling for spatial compression
- Fully connected classifier for final prediction

This structure ensures that the model learns rich feature representations, which are essential for distinguishing between different crops and diseases.

#### 4.1.1 Input Layer

The journey begins with the Input Layer, where the system accepts a crop image of size  $224 \times 224 \times 3$ . This means the image is 224 pixels wide, 224 pixels tall, and has 3 color

channels — Red, Green, and Blue (RGB). This is a standard image size used in most deep learning vision models because it provides enough detail for the model to learn from without being too large to process efficiently.

### 4.1.2 Stem Layer (Initial Feature Extraction)

Once the image enters the network, it first passes through the Stem Layer, which consists of a  $7 \times 7$  Convolution with 64 filters and a stride of 2, followed by Batch Normalization and a ReLU activation function. Think of this layer as the model's first look at the image — it scans the image with large filters to detect basic low-level features like edges, textures, and color gradients. The stride of 2 reduces the spatial size of the image, making computation faster in the layers ahead. After the stem layer, a  $3 \times 3$  Max Pooling with stride 2 is applied to further reduce the image dimensions while retaining the most important features.

### 4.1.3 Dense Block 1 (6 Layers)

The processed features then enter the first Dense Block, which contains 6 layers. What makes DenseNet special and different from regular CNNs is that every layer inside a Dense Block is connected to every other layer — meaning each layer receives the feature maps from all previous layers as input. This encourages maximum feature reuse and ensures that no important information is lost as we go deeper into the network. Each layer inside this block uses a Bottleneck Convolution structure: a  $1 \times 1$  conv followed by a  $3 \times 3$  conv, along with Batch Normalization and ReLU. The Growth Rate is set to 32, which means each layer adds 32 new feature maps to the collective knowledge of the block.

### 4.1.4 Transition Layer 1

After Dense Block 1, the network passes through Transition Layer 1. The purpose of this layer is to compress and reduce the number of feature maps and spatial dimensions before entering the next dense block. It does this using a  $1 \times 1$  Convolution followed by  $2 \times 2$  Average Pooling. This keeps the model efficient and prevents the number of feature channels from growing too large.

#### 4.1.5 Dense Block 2 (12 Layers)

The compressed features now enter Dense Block 2, which is deeper with 12 layers. It follows the same layer pattern as Dense Block 1 — dense connections, bottleneck convolutions, batch normalization, and ReLU activations — with a Growth Rate of 32. Because it has more layers, this block is capable of learning more complex and mid-level features from the crop images, such as patterns associated with disease spots, discoloration, or texture irregularities on leaves.

#### 4.1.6 Transition Layer 2

Similar to the first transition layer, Transition Layer 2 again reduces the spatial dimensions and feature map count using a  $2 \times 2$  Average Pooling operation, preparing the data for the even deeper blocks ahead.

#### 4.1.7 Dense Block 3 (48 Layers — The Deepest Block)

This is the most powerful and computationally intensive block in the entire architecture — Dense Block 3 with 48 layers. It is described as the Deepest Block with High Feature Reuse, and it maintains the same Growth Rate of 32. With 48 densely connected layers, this block dives deep into the abstract representations of the image. By this stage, the network has learned very high-level, complex patterns — things like specific disease signatures, texture anomalies, and structural differences between healthy and diseased crop leaves. The dense connections here ensure that even the earliest learned features are still accessible to the deepest layers.

#### 4.1.8 Transition Layer 3

Transition Layer 3 once again applies a  $1 \times 1$  conv and  $2 \times 2$  Average Pooling to reduce dimensions after the deepest block, passing a compact but information-rich feature set forward.

#### 4.1.9 Dense Block 4 (32 Layers — Final High-Level Feature Extraction)

The fourth and final dense block contains 32 layers and is responsible for Final High-Level Feature Extraction. At this stage, the model is refining its understanding of the image to

make a final, confident decision about what disease — if any — is present. The same layer pattern continues here, ensuring every piece of information gathered from earlier blocks is utilized effectively.

#### **4.1.10 Global Average Pooling (Spatial Feature Compression)**

After all four dense blocks and their transitions, the feature maps pass through Global Average Pooling (GAP). This layer takes the entire spatial feature map for each channel and compresses it into a single number by averaging. The result is a compact, one-dimensional feature vector that summarizes everything the network has learned about the image. This step removes spatial information that is no longer needed and prepares the features for the final classification stage.

#### **4.1.11 Classifier Head (Final Decision Maker)**

The compressed feature vector then enters the Classifier Head, which is a Fully Connected (Dense) layer with 512 neurons, followed by ReLU Activation and Dropout for regularization (to prevent overfitting). Finally, a Linear Layer maps the 512 features down to the number of output classes (`num_classes`), and a `LogSoftMax` function is applied to produce a probability

#### **4.1.12 Final Output — Crop Verification / Disease Classification**

The network produces its final prediction, classifying the input crop image into one of the following categories:

- Bacterial Spot
- Healthy
- Leaf Curl Virus
- White Spot
- Chilli
- Onion
- Wheat

## 4.2 Dataset Implementation and Preparation

### 4.2.1 Dataset Collection and Sources

The dataset used for building the Crop Health Monitoring System was carefully gathered from multiple reliable and diverse sources to ensure the model's accuracy and real-world applicability. The primary sources include Kaggle plant disease repositories, the Tamil Nadu Agricultural University (TNAU) field image collections, Medley disease repositories, and most importantly, drone-captured images taken directly from agricultural fields. By combining publicly available datasets with real field images collected from TNAU and drone footage, the system is trained on a rich and varied set of crop conditions that closely reflect actual farming environments.

### 4.2.2 Dataset Organization and Structure

The collected dataset was organized in a structured and systematic way to ensure smooth model training. Each crop is assigned its own dedicated folder, and within each crop folder, multiple sub-folders are created — one for each disease category along with a healthy class. For example, the Paddy crop folder contains sub-folders for diseases such as Bacterial Leaf Blight, Blast, Brown Spot, Sheath Blight, and a Healthy category, among others. This hierarchical folder structure makes it easy for the training pipeline to identify and separate different classes during the learning process.

### 4.2.3 Data Processing and Augmentation

To handle the dataset efficiently, a dedicated Python script was developed to automate the entire data preparation process. This script automatically splits the dataset into training and validation sets, ensuring that the model is evaluated on unseen data during training. It also applies various image augmentation techniques — such as rotation, flipping, zooming, and brightness adjustments — to artificially increase the diversity of the dataset and make the model more robust against real-world variations. Additionally, the script creates model checkpoints to save the best-performing versions of the model during training, and automatically plots accuracy and loss curves to help monitor and evaluate the training progress visually.

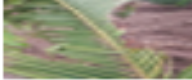
| S. No. | Crop Name | Crop Image                                                                          | Crop Disease Detection                                                                                                                                                                                                            |
|--------|-----------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | Chilli    |    | Bacterial Spot, Cercospora Leaf Spot, Healthy, Leaf Curl Virus, Nutrition Deficiency, White Spot                                                                                                                                  |
| 2      | Coconut   |    | Bud Root Dropping, Bud Rot, Gray Leaf Spot, Leaf Rot, Stem Bleeding, Yellowing                                                                                                                                                    |
| 3      | Corn      |    | Blight, Common Rust, Gray Leaf Spot, Healthy                                                                                                                                                                                      |
| 4      | Cotton    |    | Bacterial Blight, Curl Virus, Healthy, Herbicide Growth Damage, Leaf Hopper Jassids, Leaf Redding, Leaf Variegation, Powdery Mildew                                                                                               |
| 5      | Maize     |    | Cercospora Leaf Spot, Common Rust, Fall Armyworm, Grasshopper, Healthy, Leaf Beetle, North Leaf Blight, Streak Virus                                                                                                              |
| 6      | Onion     |   | Alternaria, Bulb Blight, Fusarium, Healthy, Virosis                                                                                                                                                                               |
| 7      | Paddy     |  | Bacterial Leaf Blight, Bacterial Leaf Streak, Bacterial Panicle Blight, Bakanae, Blast, Dead Heart, False Smut or Lakshmi, Grain Discolouration, Healthy, Hipsa, Rice Grassy stunt, Rice Tungro, Rice Yellow Dwarf, Sheath Blight |
| 8      | Sugarcane |  | Healthy, Mosaic, Pokkah Boeng, Ratoon Stunting, Red Rot, Rust, Sett Rot, Smut, Wilt, Yellow Leaf Disease                                                                                                                          |
| 9      | Tomatoes  |  | Bacterial Spot, Early Blight, Healthy, Late Blight, Leaf Mold, Mosaic Virus, Septoria Leaf Spot, Spider Mites, Target Spot, Yellow Leaf Curl Virus                                                                                |
| 10     | Wheat     |  | Healthy, Leaf Rust or Brown Rust, Loose Smut, Powdery Mildew, Stem Rust, Stripe Rust or Yellow Rust                                                                                                                               |

FIGURE 4.2: Representative Dataset for Each Crop and Disease Classes

#### 4.2.4 Selected Crops and Image Distribution

A total of Ten crops were carefully selected to build a comprehensive and practical crop health monitoring system. These crops represent some of the most widely cultivated and economically important agricultural produce in India. The selected crops along with

their respective image counts are as follows — Chili with 1,856 images, Coconut with 7,005 images, Corn with 3,156 images, Cotton with 7,005 images, Maize with 6,913 images, Onion with 2,165 images, Paddy with 10,837 images, Sugarcane with 4,179 images, Tomatoes with 18,345 images, and Wheat with 8,369 images. Together, these ten crops form a large and diverse dataset that covers a wide spectrum of disease types and growth conditions across different agricultural settings.

### 4.3 Training Workflow of the Model

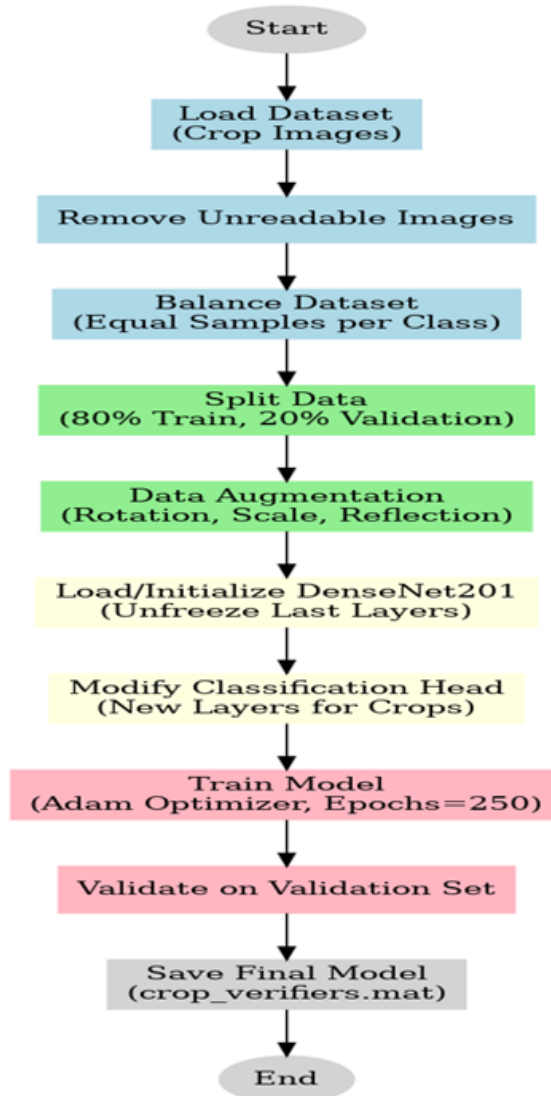


FIGURE 4.3: Workflow of the Model Training

The training process follows a structured workflow, as illustrated in the provided above flowchart. The process begins with loading the dataset and preparing it for training. After preprocessing and augmentation, the model is initialized using a pre-trained DenseNet architecture.

The final layers of the model are modified to match the number of crop and disease classes in the dataset. This process, known as transfer learning, allows the model to leverage previously learned features while adapting to the specific problem.

The model is trained using the Adam optimizer for a fixed number of epochs. During training, the model learns to minimize classification error by adjusting its weights. Validation is performed after each epoch to monitor performance and prevent overfitting.

Finally, the trained model is saved for deployment. This structured workflow ensures efficient training and reliable performance.

## 4.4 Crop Verification Model

The crop verification model is specifically designed to identify the crop type from the input image. This model ensures that the correct crop is selected before performing disease detection.

The model is trained using images of different crop types, enabling it to distinguish between crops based on visual characteristics. By automating crop selection, the system reduces user dependency and minimizes errors.

The output of this model is used as an input parameter for the disease detection model, ensuring that disease classification is performed accurately for the correct crop.

## 4.5 Disease Detection Model

The disease detection model is responsible for identifying the type of disease affecting the crop. This model is trained on a dataset containing multiple disease categories along with healthy samples.

The model analyzes features such as leaf discoloration, texture variation, and structural changes. Based on these features, it classifies the crop condition into different categories such as bacterial infection, viral infection, nutrient deficiency, or healthy.

This model forms the core of the system, as it provides the primary diagnosis of crop health.

## 4.6 ONNX Model Export and Deployment

After training, the model needs to be converted into a format suitable for deployment in the Android application. For this purpose, the model is exported into ONNX (Open Neural Network Exchange) format.

ONNX provides a standardized representation of machine learning models, enabling interoperability between different frameworks. The trained model is converted into ONNX format using appropriate export functions.

The ONNX model includes:

- Model architecture
- Learned weights
- Input and output specifications

Once exported, the ONNX model is integrated into the Android application using ONNX Runtime. The application loads the model, preprocesses input images, and performs inference in real time.

Special care is taken to ensure:

- Correct input tensor shape
- Proper normalization of image data
- Efficient memory management

This deployment strategy ensures that the model runs efficiently on mobile devices without requiring high computational resources.

## Chapter 5

# Development of Crop Health Monitoring System Android Application

The Android application serves as the primary interface between the user and the Crop Health Monitoring System. It is designed to provide a simple, efficient, and interactive platform through which users can upload crop images, analyze crop health, and visualize results in real time. The application integrates multiple components such as image processing, machine learning inference, database management, and graphical visualization into a single cohesive system.

The development of the application focuses on usability, performance, and seamless integration of AI models. It ensures that users, especially farmers and agricultural practitioners, can operate the system without requiring technical expertise.

### 5.1 Development Environment and Technologies Used

The application is developed using modern Android development tools and technologies. The implementation is carried out using Android Studio as the integrated development environment.

The primary programming language used is Kotlin, which provides concise syntax, improved safety features, and better support for asynchronous programming through coroutines.

The system utilizes several libraries and frameworks, each serving a specific purpose:

- TensorFlow Lite / ONNX Runtime: Used for running trained AI models on the mobile device for crop verification and disease detection.
- OpenCV (optional): Used for image preprocessing and HSV conversion if implemented at a lower level.
- MPAndroidChart: Used for generating graphical representations of crop health trends.
- Room Database: Used for local data storage and management of crop analysis results.
- Coroutines (Kotlin): Used for background processing to ensure smooth UI performance.
- Glide / Bitmap Processing APIs: Used for handling image loading and manipulation.

These technologies are integrated to ensure efficient performance and real-time processing within the application.

## 5.2 User Interface Module Design

The user interface of the application is designed using XML layouts, ensuring a structured and responsive design. The interface is divided into multiple sections, including header, input controls, result display, and visualization.

The top section of the application includes institutional information along with logos, providing a professional appearance. Below this, input components such as crop selection and spray stage selection are provided.

Buttons are included for:

- Uploading drone images

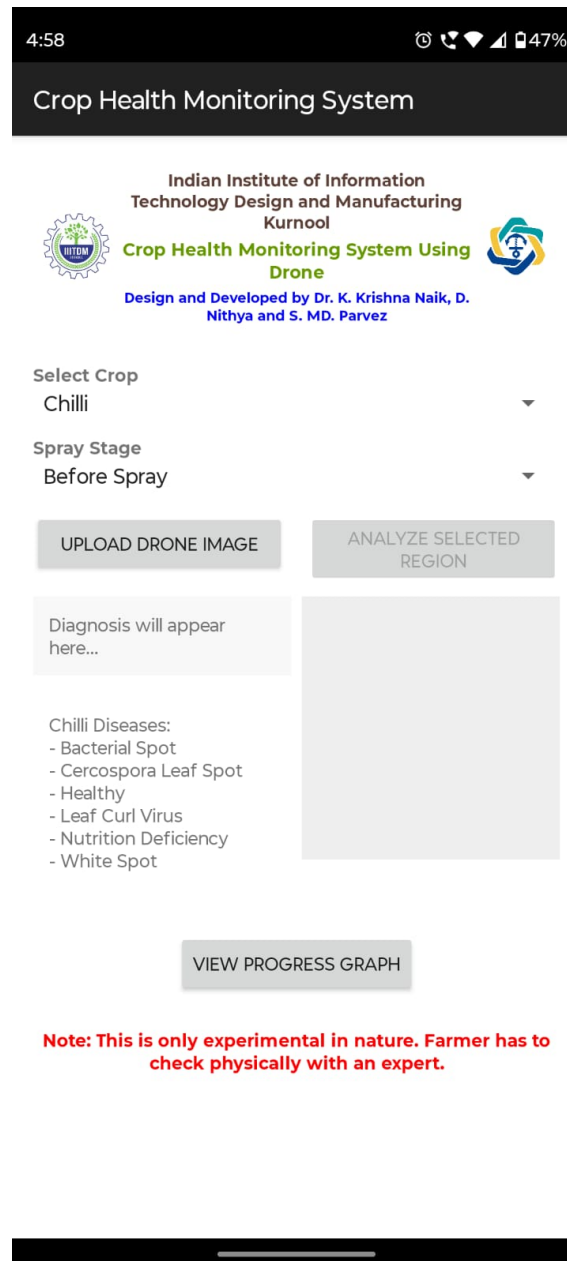


FIGURE 5.1: Main User Interface of Crop Health Monitoring System Application

- Analyzing selected regions
- Viewing progress graphs

The design ensures that all functionalities are easily accessible and clearly visible to the user.

### 5.3 Image Upload and Processing Module

The image upload module allows users to capture images using the camera or select images from the gallery. Once the image is selected, it is displayed on the screen for further analysis.

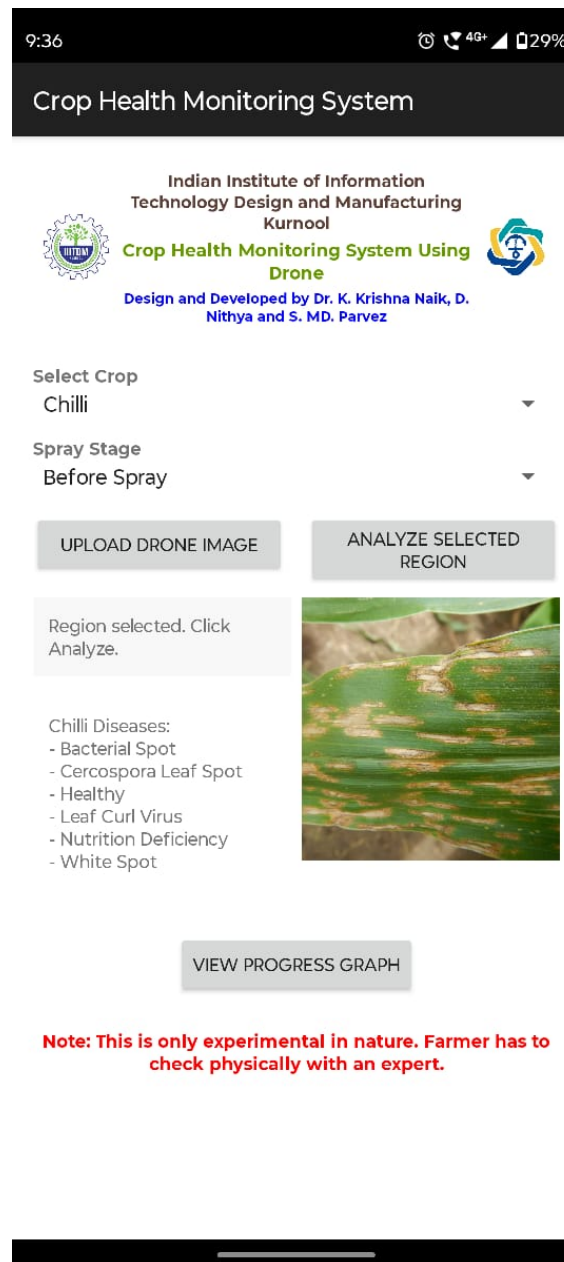


FIGURE 5.2: Image Upload and Preview Interface

The application processes the image by resizing it and converting it into a suitable format required by the AI model. Bitmap processing is used to handle image data efficiently.

This module ensures that the input image is properly prepared before being passed to the crop verification and disease detection models.

## 5.4 Crop Verification and Disease Detection Trained Models Integration

The application integrates trained AI models using TensorFlow Lite or ONNX Runtime. These models are loaded within the application and used for real-time inference.

When the user clicks the analyze button, the application performs the following steps internally:

- The image is passed to the crop verification model
- The detected crop is validated
- The same image is then passed to the disease detection model

The results obtained from the models are displayed on the screen, including disease name and health status.

This integration enables automated decision-making and eliminates manual intervention.

## 5.5 Severity Estimation and Result Display

After disease detection, the application performs severity estimation using HSV-based image processing. The affected and healthy percentages are calculated and displayed in the result section as shown in figure 5.3.

The result section presents:

- Disease name
- Spray stage
- Affected percentage
- Healthy percentage

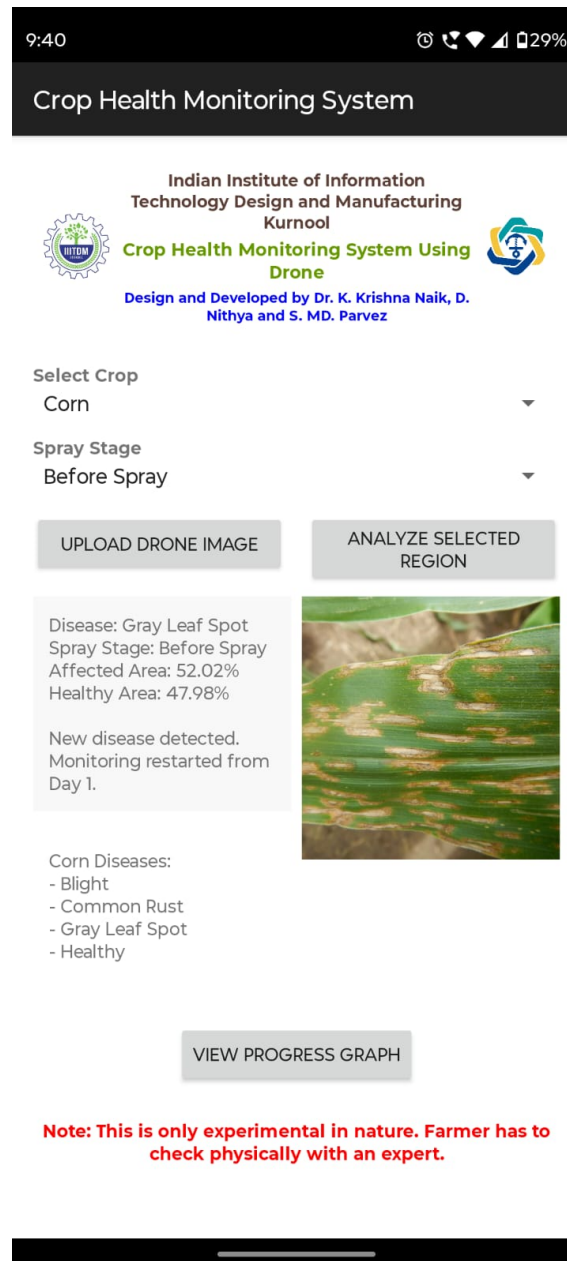


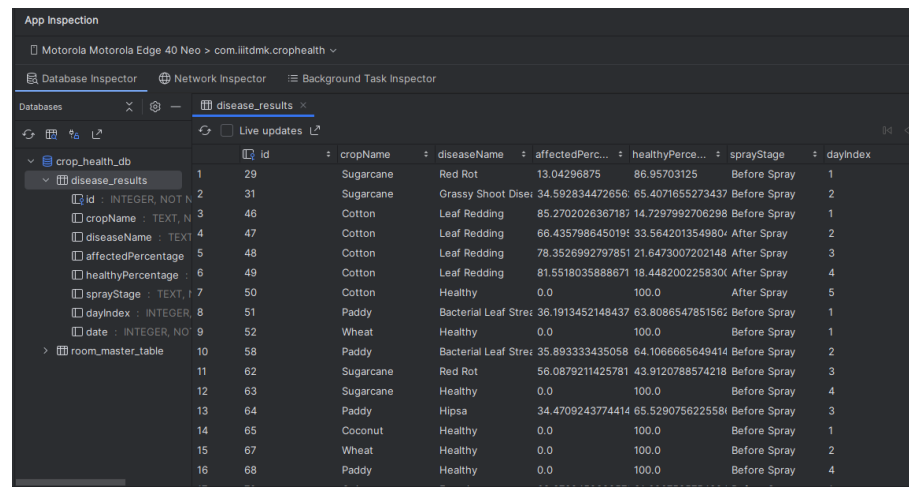
FIGURE 5.3: Disease Detection Output Display

- Trend message in View Progress Graph

This structured output helps users easily understand the condition of the crop.

## 5.6 Database Implementation using Room

The application uses Room Database for storing crop analysis results. This database allows efficient storage and retrieval of historical data.



| id | cropName  | diseaseName           | affectedPercentage | healthyPercentage | sprayStage   | dayIndex |
|----|-----------|-----------------------|--------------------|-------------------|--------------|----------|
| 1  | Sugarcane | Red Rot               | 13.04296875        | 86.95703125       | Before Spray | 1        |
| 2  | Sugarcane | Grassy Shoot Disease  | 34.592834472656    | 65.4071655273437  | Before Spray | 2        |
| 3  | Cotton    | Leaf Redding          | 85.2702026367187   | 14.7297992706298  | Before Spray | 1        |
| 4  | Cotton    | Leaf Redding          | 66.435798645018    | 33.564201354980   | After Spray  | 2        |
| 5  | Cotton    | Leaf Redding          | 78.3526992797851   | 21.6473007202148  | After Spray  | 3        |
| 6  | Cotton    | Leaf Redding          | 81.5518035888671   | 18.448200225830   | After Spray  | 4        |
| 7  | Cotton    | Healthy               | 0.0                | 100.0             | After Spray  | 5        |
| 8  | Paddy     | Bacterial Leaf Streak | 36.1913452148437   | 63.8086547851562  | Before Spray | 1        |
| 9  | Wheat     | Healthy               | 0.0                | 100.0             | Before Spray | 1        |
| 10 | Paddy     | Bacterial Leaf Streak | 35.893333435058    | 64.1066665649414  | Before Spray | 2        |
| 11 | Sugarcane | Red Rot               | 56.0879211425781   | 43.9120788574218  | Before Spray | 3        |
| 12 | Sugarcane | Healthy               | 0.0                | 100.0             | Before Spray | 4        |
| 13 | Paddy     | Hipsa                 | 34.4709243774414   | 65.5290756225586  | Before Spray | 3        |
| 14 | Coconut   | Healthy               | 0.0                | 100.0             | Before Spray | 1        |
| 15 | Wheat     | Healthy               | 0.0                | 100.0             | Before Spray | 2        |
| 16 | Paddy     | Healthy               | 0.0                | 100.0             | Before Spray | 4        |

FIGURE 5.4: Crop Health Database

Each entry in the database contains:

- Crop name
- Disease type
- Affected percentage
- Healthy percentage
- Spray stage
- Date and time

The database is accessed using Data Access Objects (DAO), which provide structured queries for inserting and retrieving data.

This stored data is later used for generating graphs and analyzing trends over time.

## 5.7 Graph Visualization Module

The graph visualization module is implemented using MPAndroidChart. This module generates a line graph that represents crop health trends over multiple days.

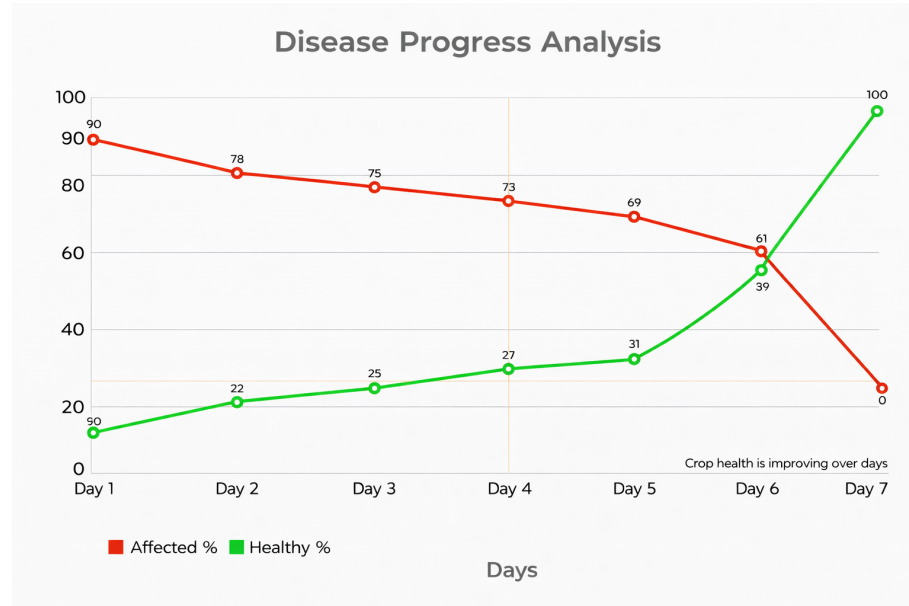


FIGURE 5.5: Crop Health Trend Graph Visualization

The graph includes:

- X-axis representing days
- Y-axis representing percentage values

Two lines are plotted:

- Affected percentage
- Healthy percentage

The graph dynamically updates as new data is added to the database.

## 5.8 Background Processing and Performance Optimization

To ensure smooth application performance, background processing is implemented using Kotlin coroutines. Heavy tasks such as model inference and database operations are executed in background threads.

This prevents UI freezing and ensures a responsive user experience. Efficient memory management techniques are also applied to handle large image data.

## Chapter 6

# Results and Discussions

The following results and discussions have been carried out after implementing and testing various components of the Crop Health Monitoring System using Drone. Multiple experiments were performed to evaluate the performance of crop verification, disease detection, severity estimation, and graph-based monitoring.

The system was tested using different crop images under varying conditions such as lighting, background variations, and disease stages. The obtained results demonstrate that the system is capable of accurately identifying crop diseases, estimating severity, and monitoring crop health over time.

- The mobile application interface was successfully developed to allow users to upload images and analyze crop conditions. The result interface is shown in Figure 5.1.
- The crop verification and disease detection models were integrated and tested with real crop images. The detection results are shown in Figure 5.2.
- The severity estimation using HSV analysis was implemented, providing accurate affected and healthy percentages. The output is shown in Figure 5.3.
- The spray stage comparison (before and after spray) was evaluated, showing improvement in crop health over time.
- The graph visualization module was developed to track crop health trends, as shown in Figure 3.3.

## 6.1 Model Performance Evaluation

The proposed Crop Health Monitoring System using Drone was thoroughly evaluated using multiple crop-specific disease classification models along with a dedicated crop verification model. These models were developed using the DenseNet-201 architecture and trained on datasets collected from reliable sources such as TNAU, Kaggle, Medley datasets, as well as real-time drone imagery. The crop verification model plays an important role in ensuring that the uploaded image actually belongs to the selected crop before performing disease analysis. This model achieved a very high validation accuracy of 99%, which confirms its reliability in filtering incorrect inputs and improving overall system performance.

The crop-specific disease detection models also showed strong performance, with validation accuracies ranging between 89% and 99%. It was observed that crops like Tomato, Corn, and Chilli achieved higher accuracy because their disease patterns are more visually distinct and easier for the model to differentiate. On the other hand, crops such as Paddy and Sugarcane showed slightly lower accuracy due to similarities between disease classes and a larger number of categories, which makes classification more challenging. Despite these variations, the overall average accuracy of the system was around 94%, indicating that the model performs consistently well across different crop types.

The Android application was tested on multiple devices to evaluate real-time performance. The integration of ONNX Runtime Mobile enabled efficient model inference directly on the device. The prediction time for each image ranged approximately between 60 ms and 110 ms, which is suitable for real-time applications. The system also demonstrated stable performance under different conditions such as changes in lighting, image quality, and camera resolution, making it practical for real-world usage.

To further validate the system, field-level testing was conducted using 110 drone-captured images. The system achieved an accuracy between 88% and 92% in these real-world conditions, confirming its capability to detect diseases effectively even from aerial images. In addition to disease detection, the system successfully performed severity estimation using HSV-based image analysis. It calculated the percentage of affected and healthy regions and displayed the results clearly to the user.

Another important feature of the system is its ability to track crop health over time using graphical analysis. By comparing "Before Spray" and "After Spray" results, the system provides meaningful insights into disease progression. It can indicate whether the

crop condition is improving, worsening, or remaining stable. This feature is particularly useful for farmers to evaluate the effectiveness of pesticide treatments and take appropriate actions.

Overall, these results clearly demonstrate that the proposed system is efficient, lightweight, and reliable for real-time crop health monitoring. It provides accurate analysis, quick predictions, and useful decision support, making it highly suitable for practical agricultural applications.

The following figures present the disease detection results obtained through the graphical user interface implemented in MATLAB.

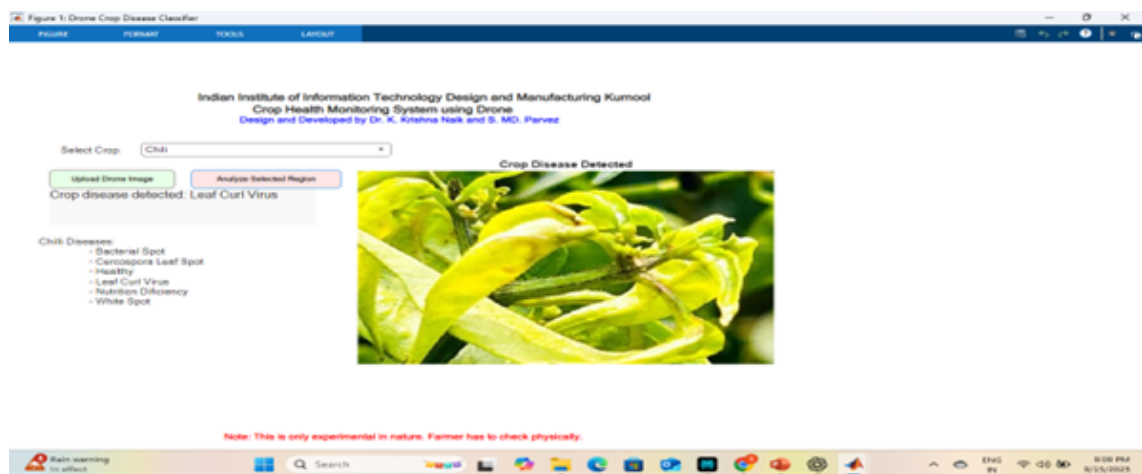


FIGURE 6.1: Chilli Crop Disease Detected

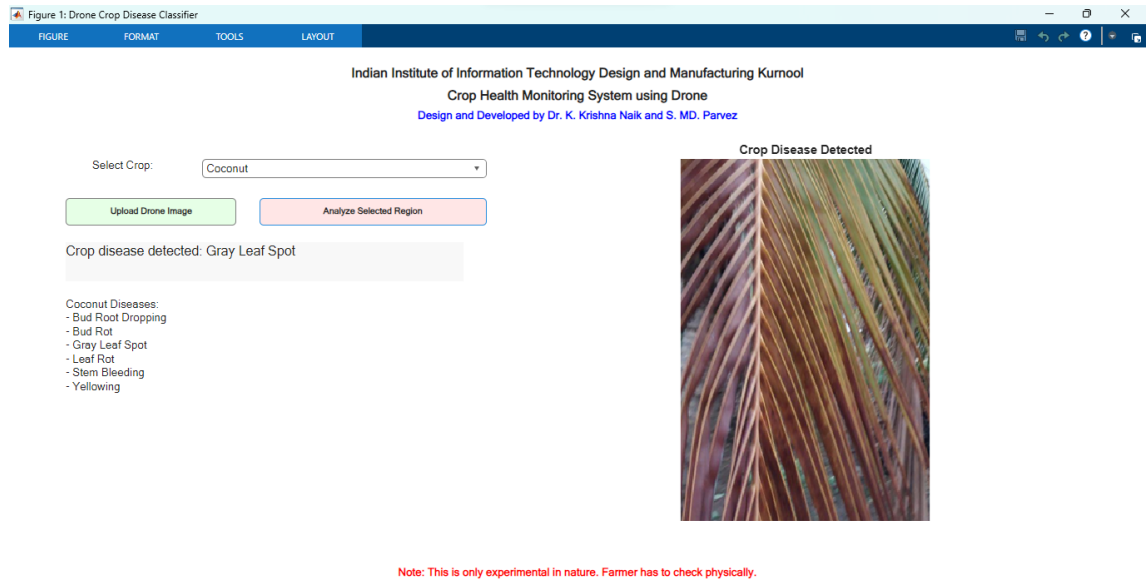


FIGURE 6.2: Coconut Crop Disease Detected

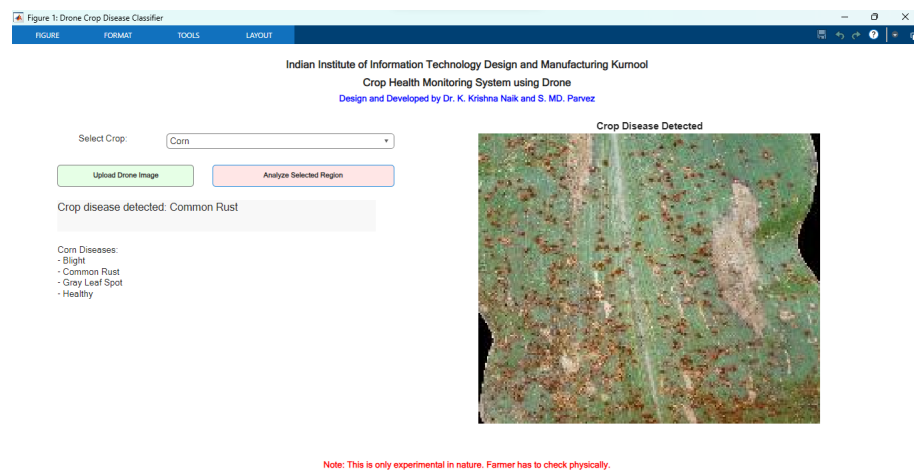


FIGURE 6.3: Corn Crop Disease Detected

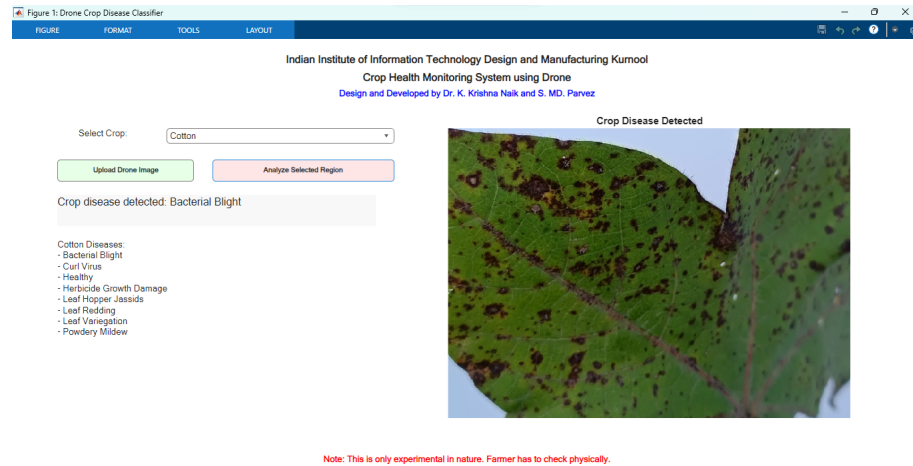


FIGURE 6.4: Cotton Crop Disease Detected

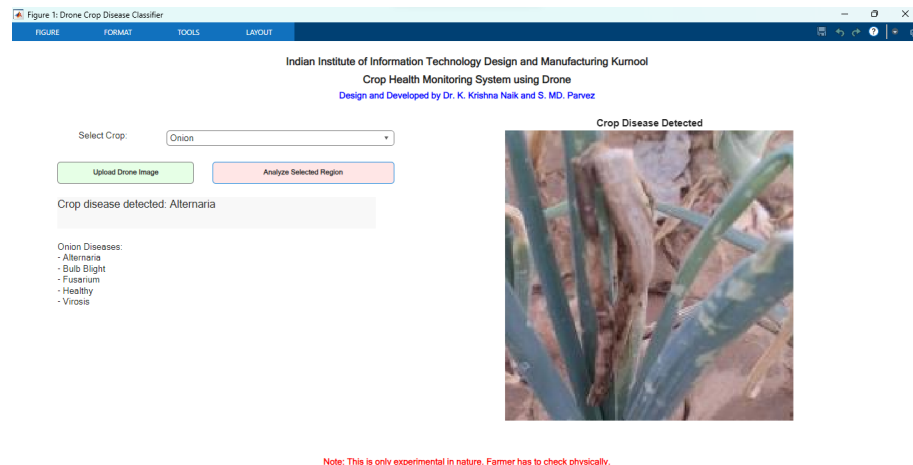


FIGURE 6.5: Onion Crop Disease Detected

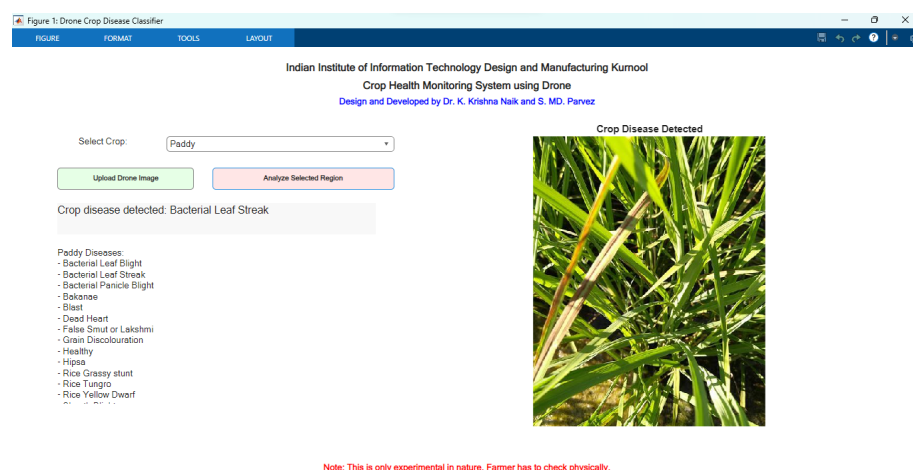


FIGURE 6.6: Paddy Crop Disease Detected

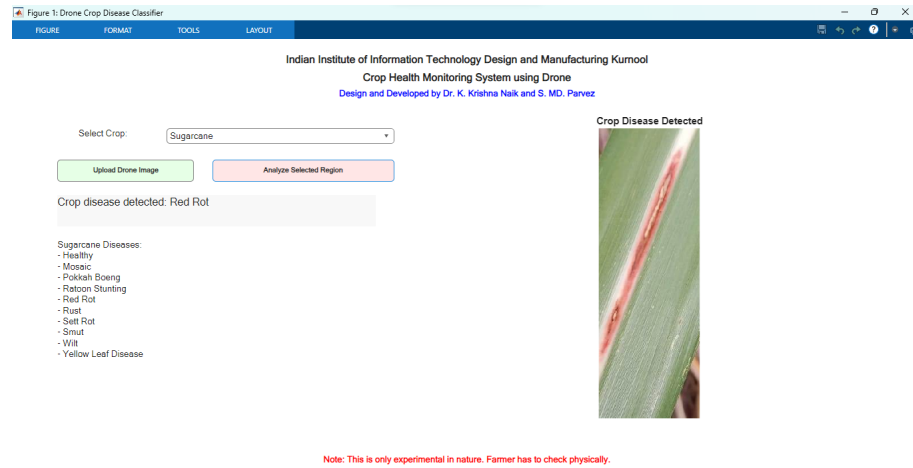


FIGURE 6.7: Sugarcane Crop Disease Detected

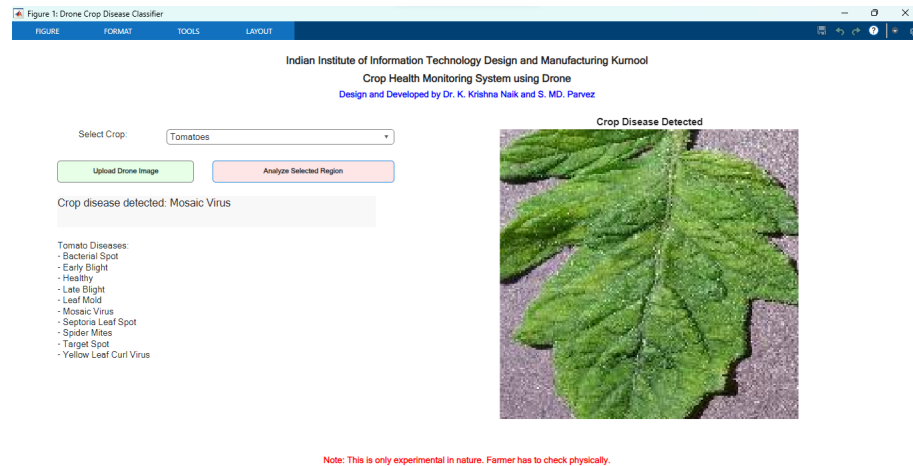


FIGURE 6.8: Tomatoes Crop Disease Detected

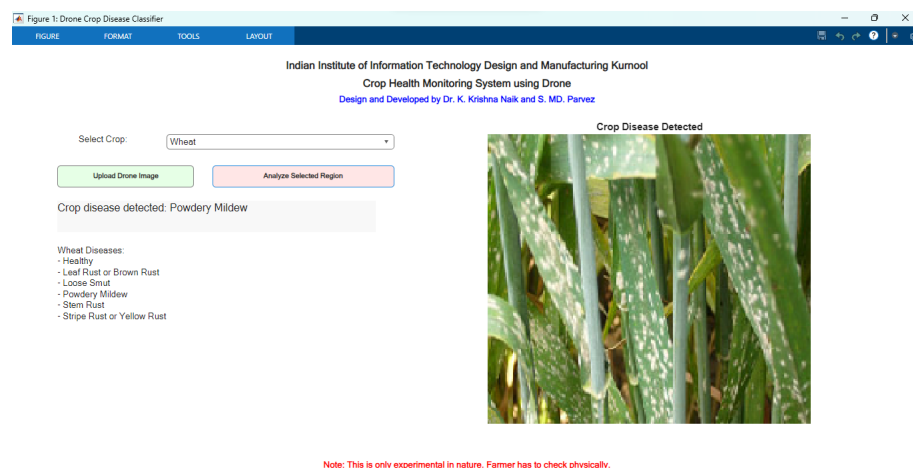


FIGURE 6.9: Wheat Crop Disease Detected



FIGURE 6.10: Crop Verification Failed (Incorrect Crop Uploaded)

The following figures present the disease detection results obtained through the Android application using onnx runtime.

### For chilli crop

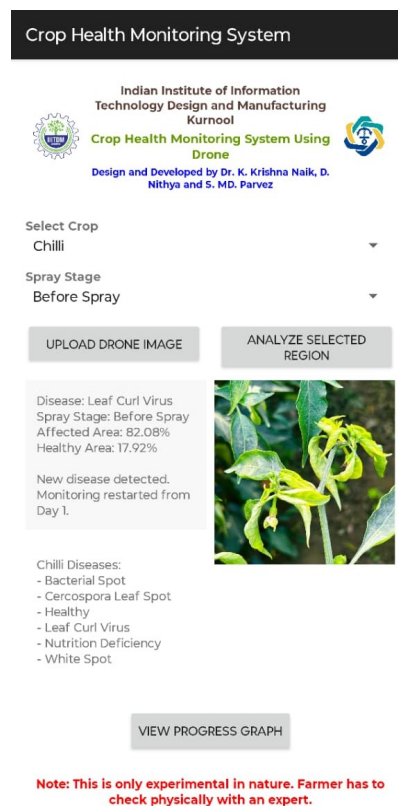


FIGURE 6.11: Before Spray Disease Detection

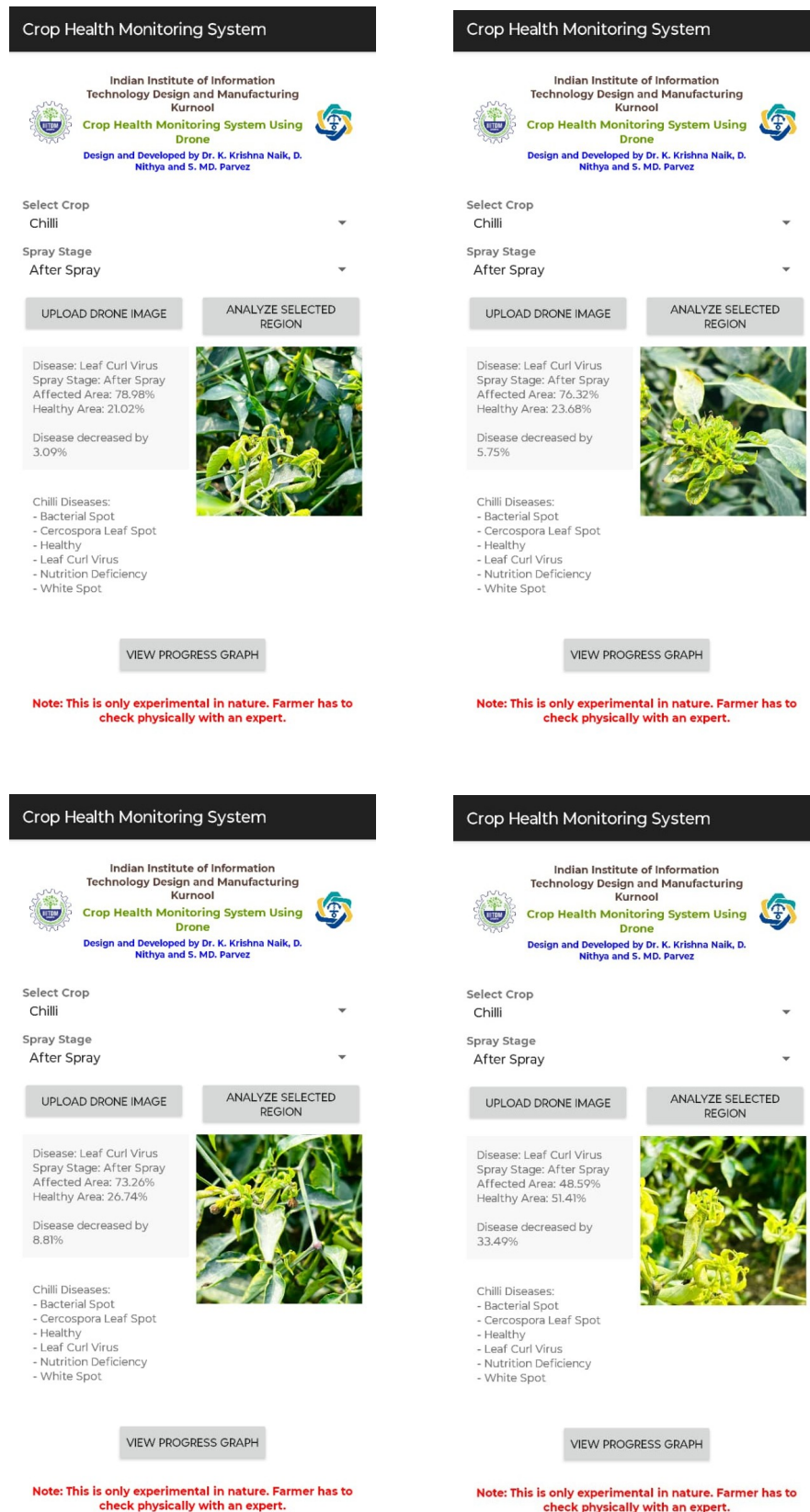


FIGURE 6.12: After Spray Disease Detection

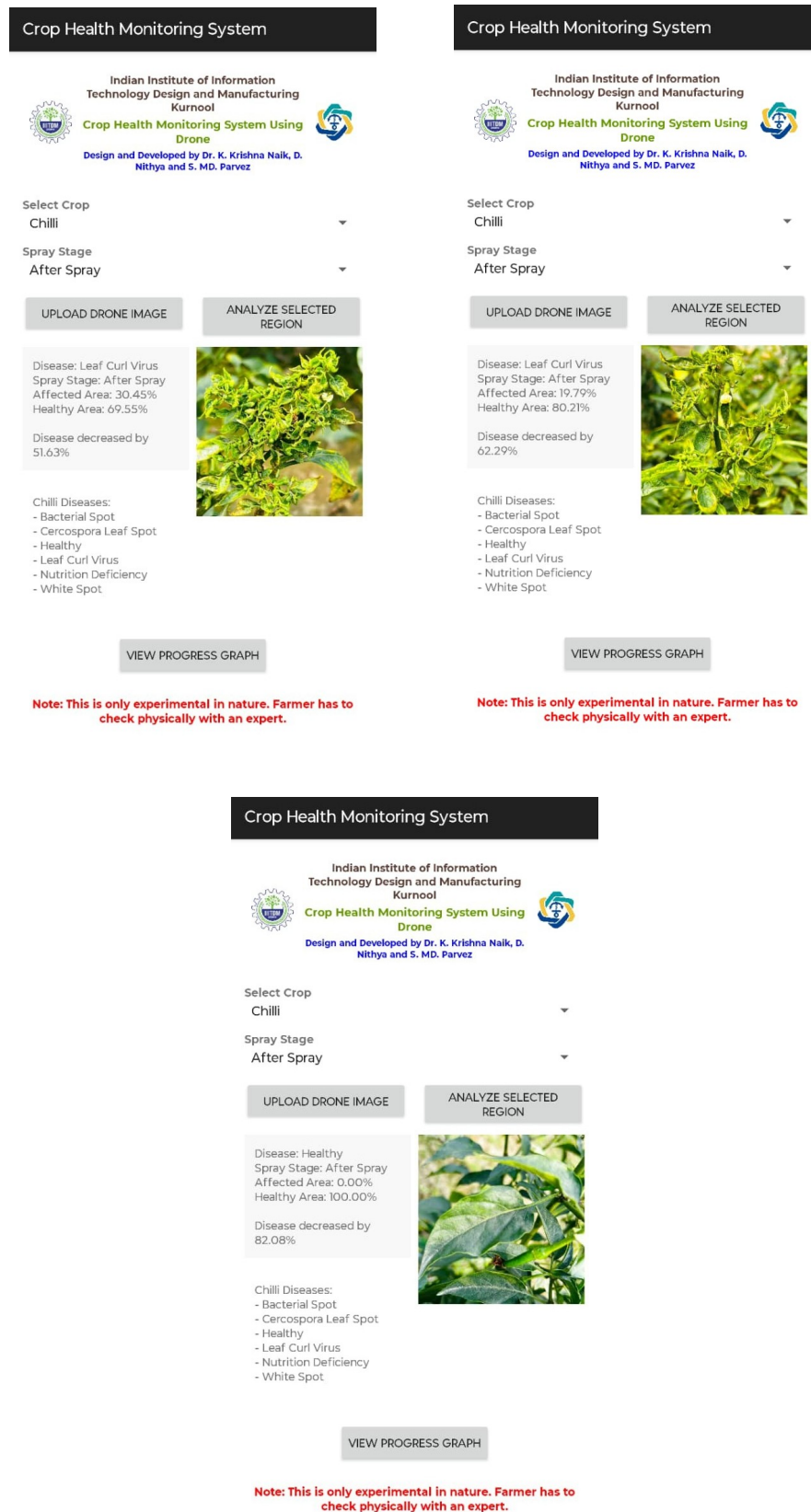


FIGURE 6.13: After Spray Disease Detection

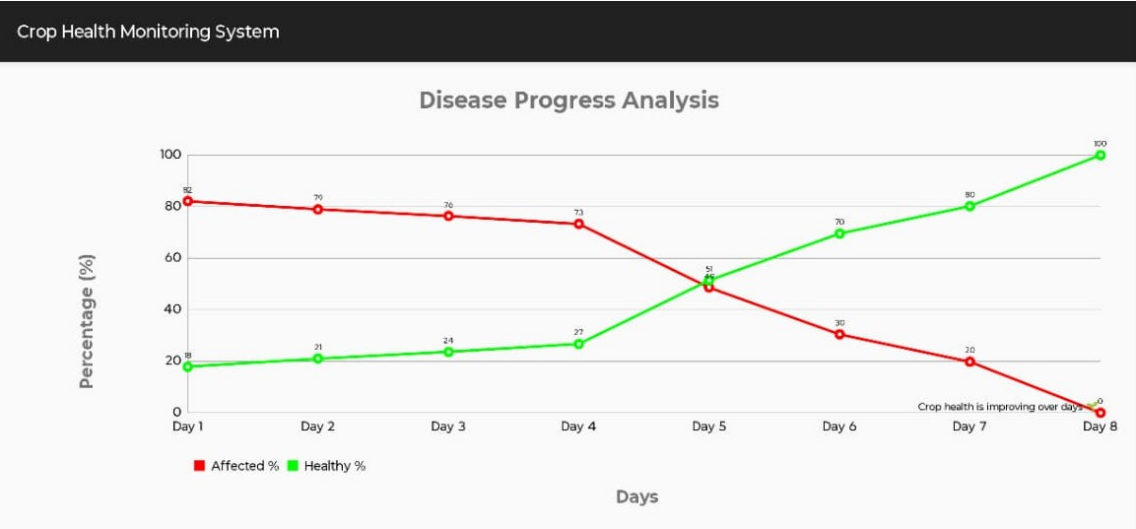


FIGURE 6.14: Disease Progress Analysis

The graphical analysis illustrates the variation of affected and healthy percentages over consecutive days, enabling clear visualization of disease progression and recovery trends.

## For Coconut crop

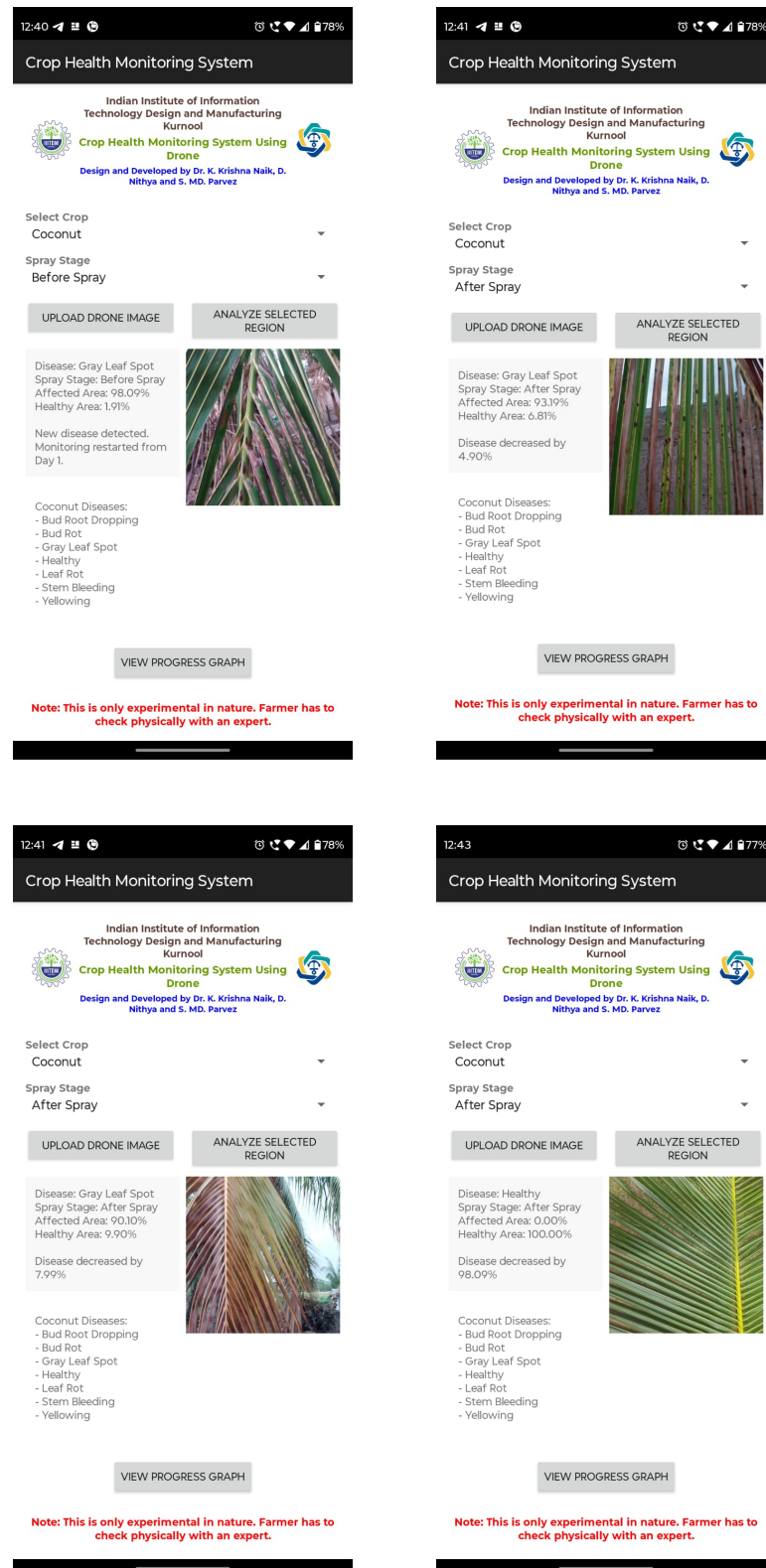


FIGURE 6.15: Before and After Spray Disease Detection

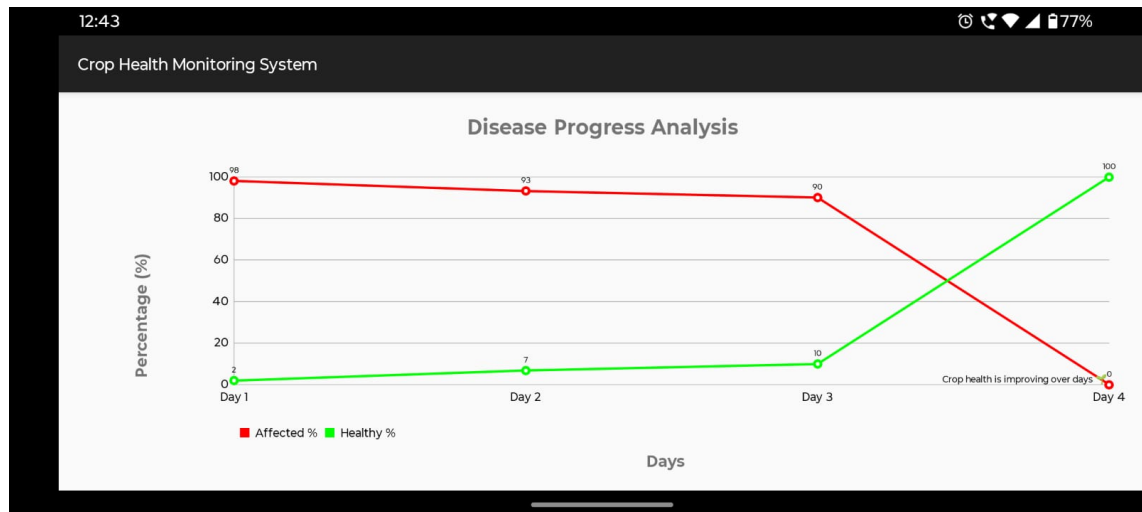


FIGURE 6.16: Disease Progress Analysis

### For Corn Crop

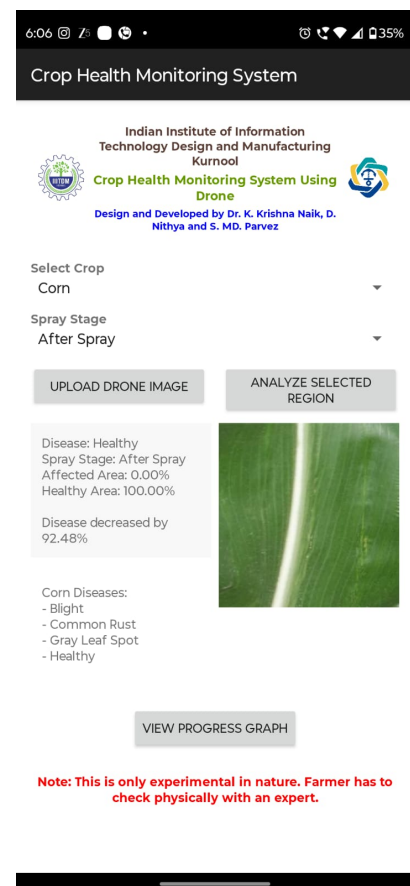
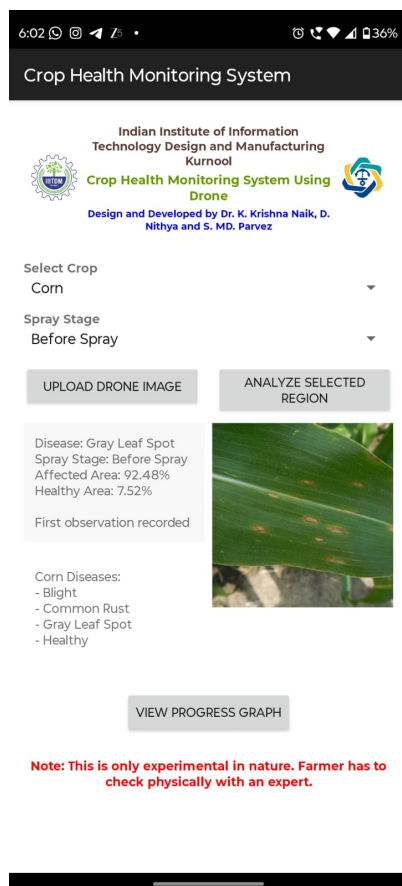


FIGURE 6.19: Before and After Spray Disease Detection

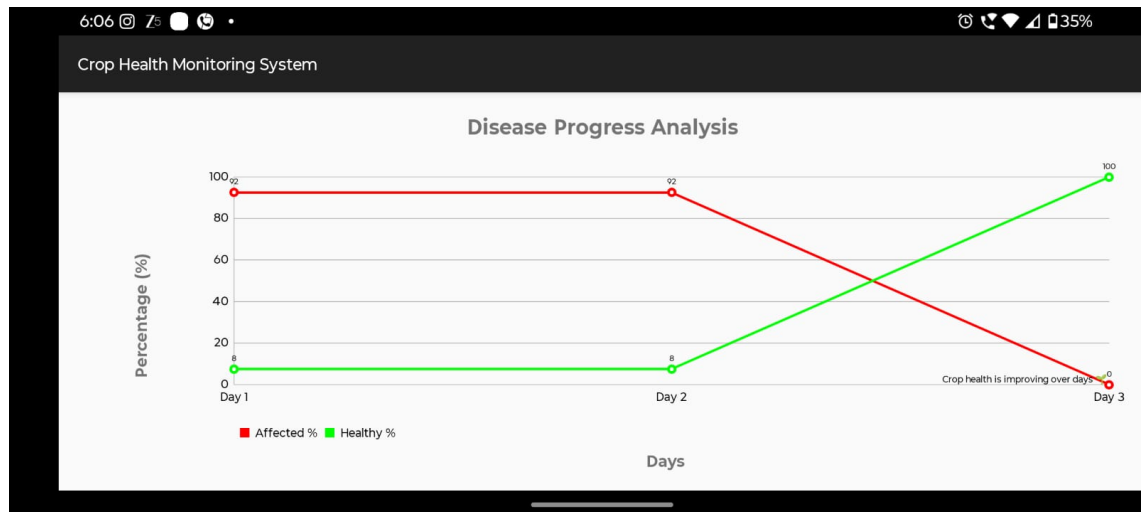


FIGURE 6.20: Disease Progress Analysis

### For Cotton Crop

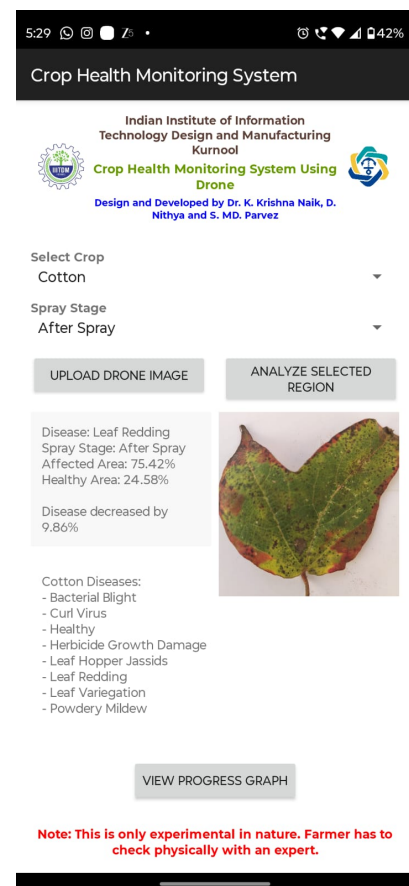
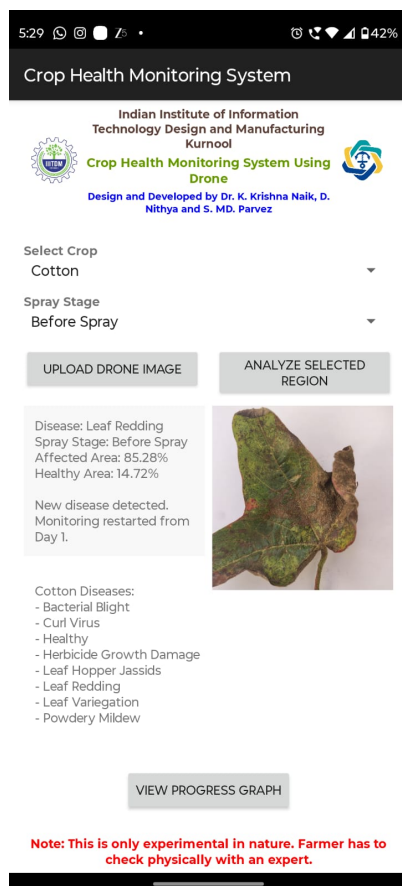


FIGURE 6.23: Before and After Spray Disease Detection

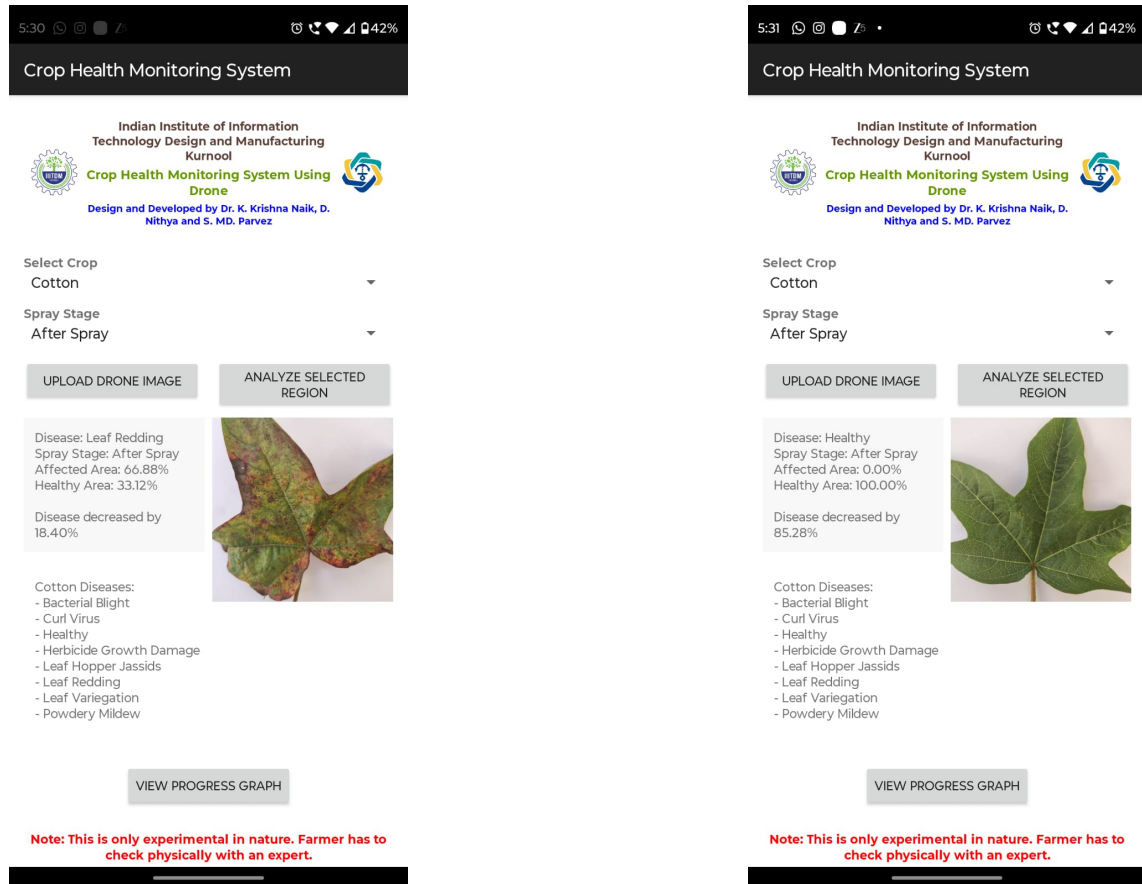


FIGURE 6.26: Before and After Spray Disease Detection

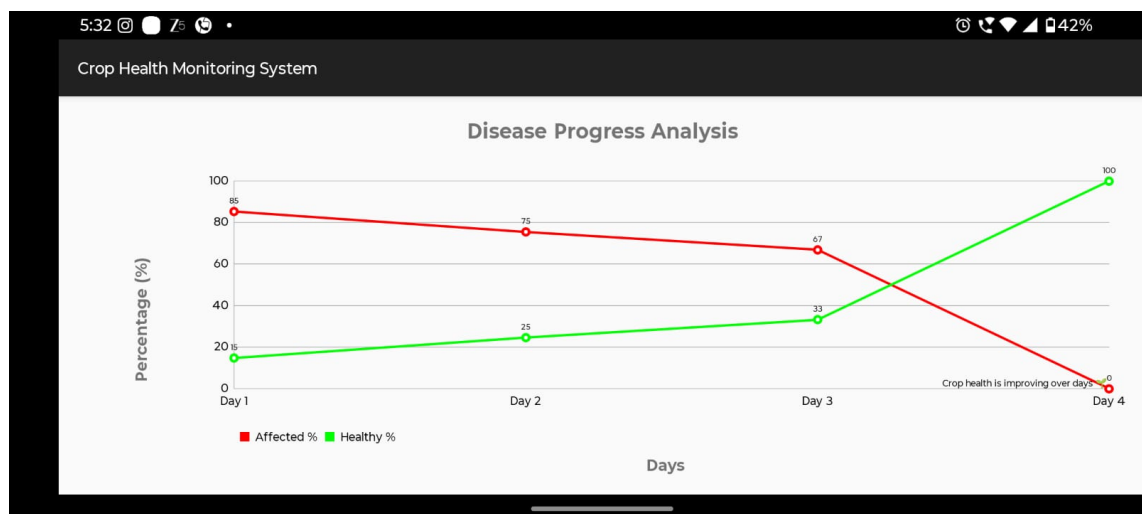


FIGURE 6.27: Disease Progress Analysis

## For Maize Crop

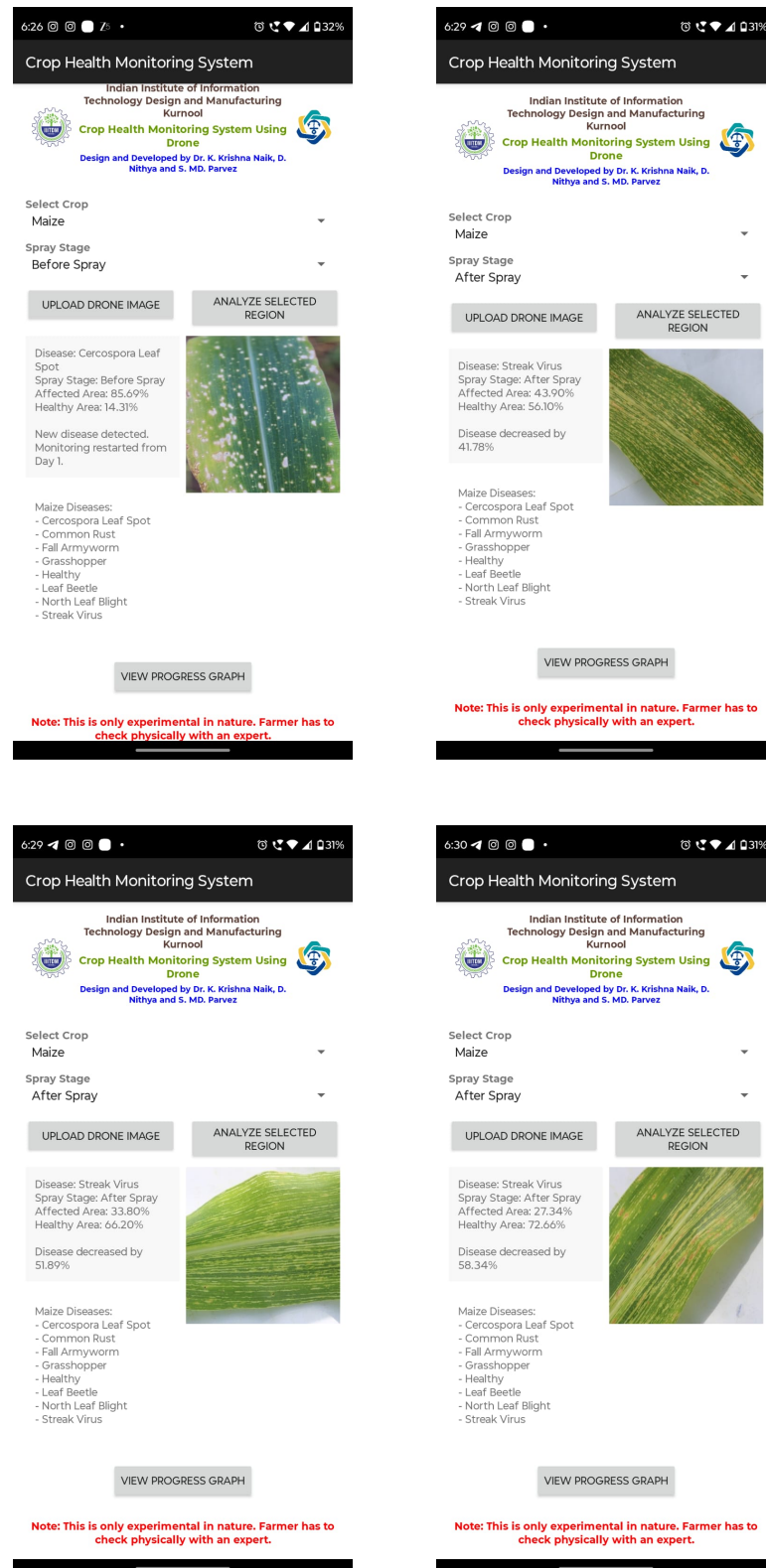


FIGURE 6.28: Before and After Spray Disease Detection

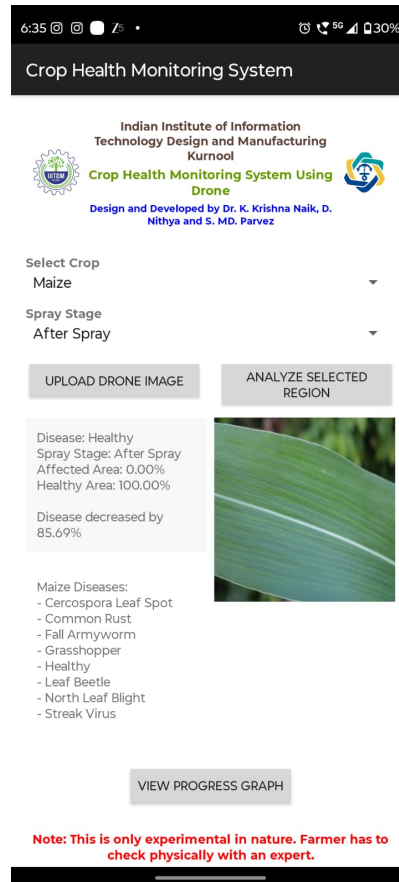


FIGURE 6.29: After Spray Disease Detection

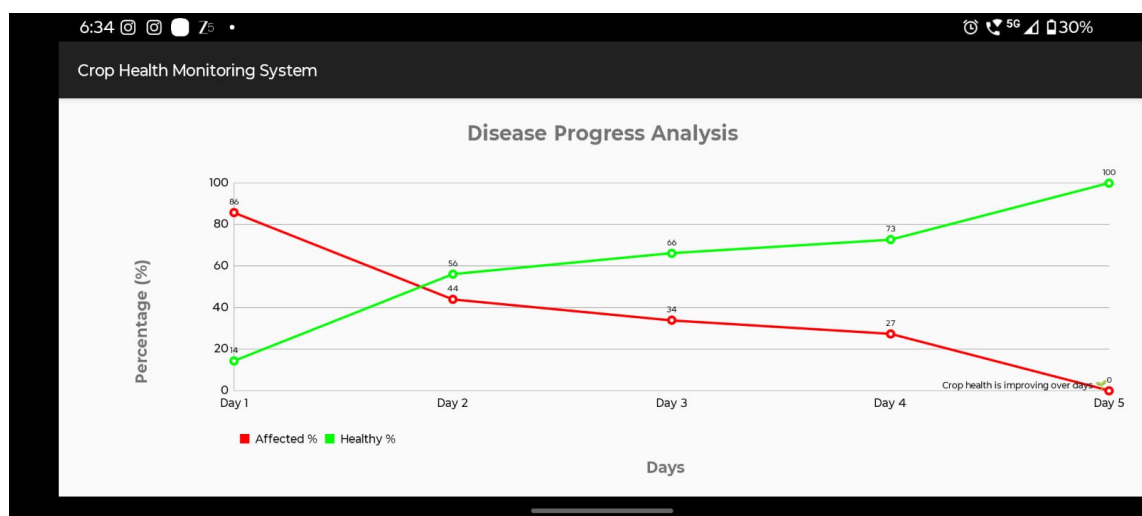


FIGURE 6.30: Disease Progress Analysis

## For Onion Crop

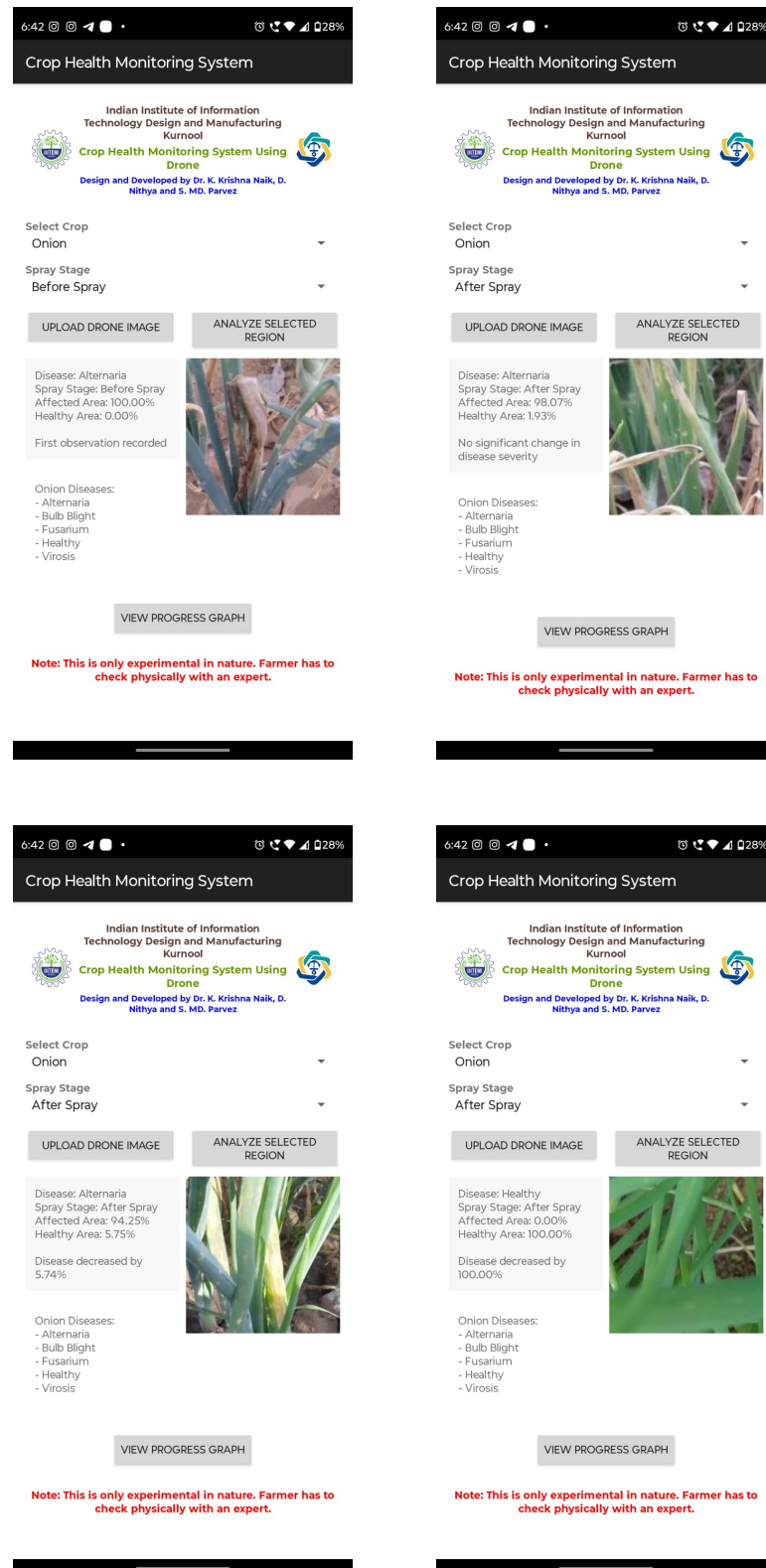


FIGURE 6.31: Before and After Spray Disease Detection

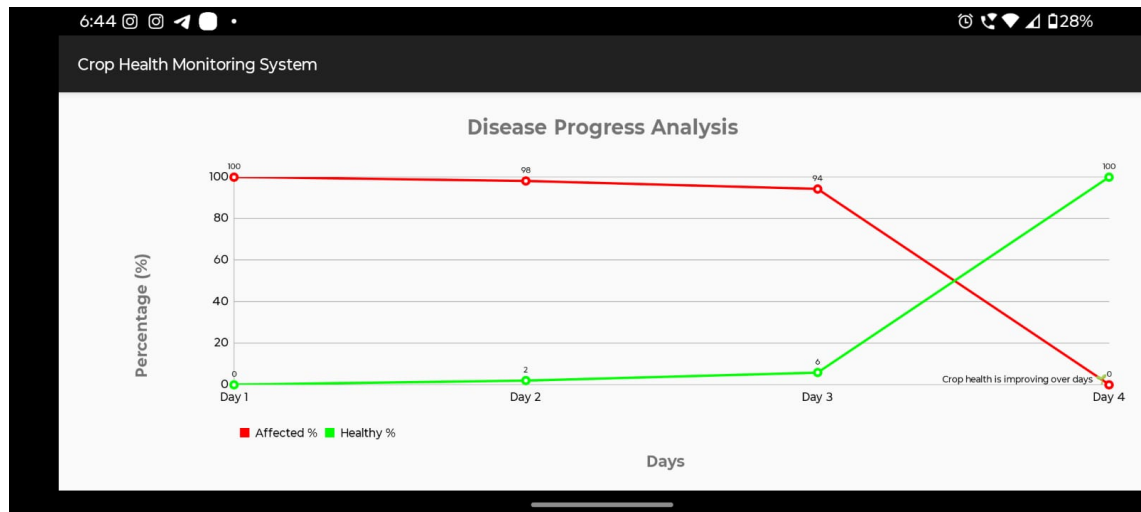


FIGURE 6.32: Disease Progress Analysis

### For Paddy Crop

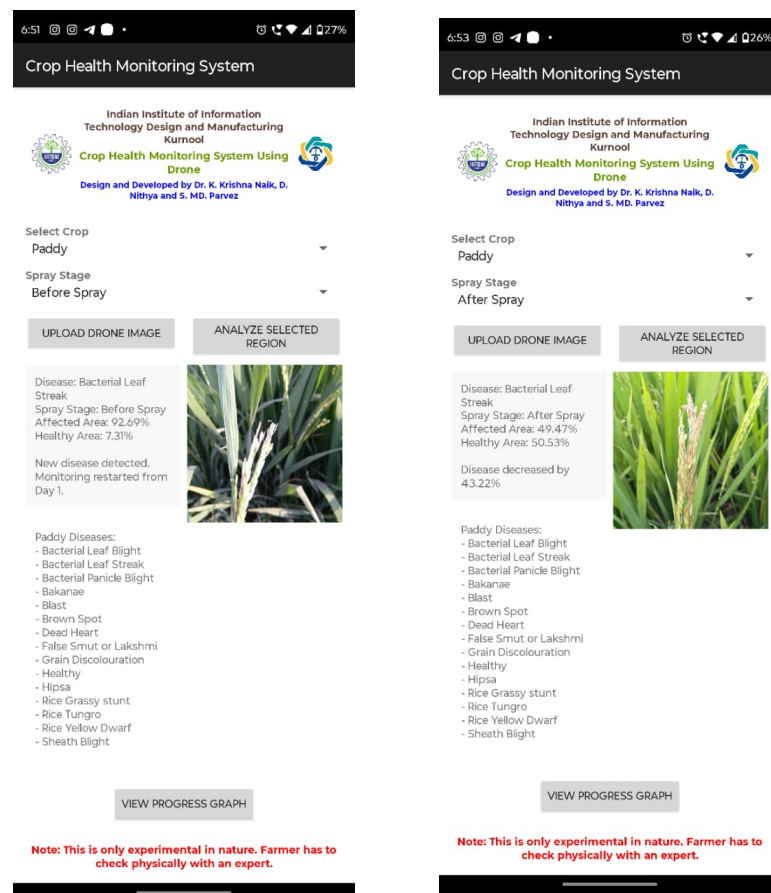
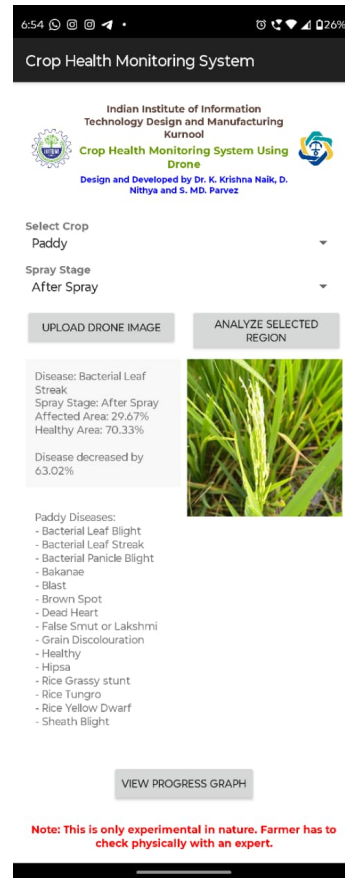
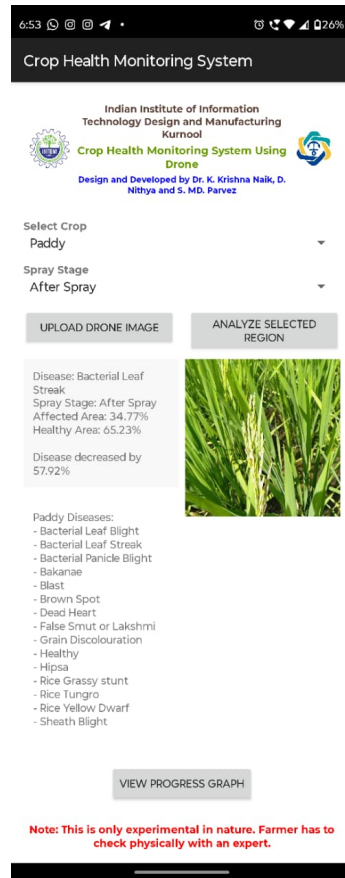


FIGURE 6.33: Before and After Spray Disease Detection



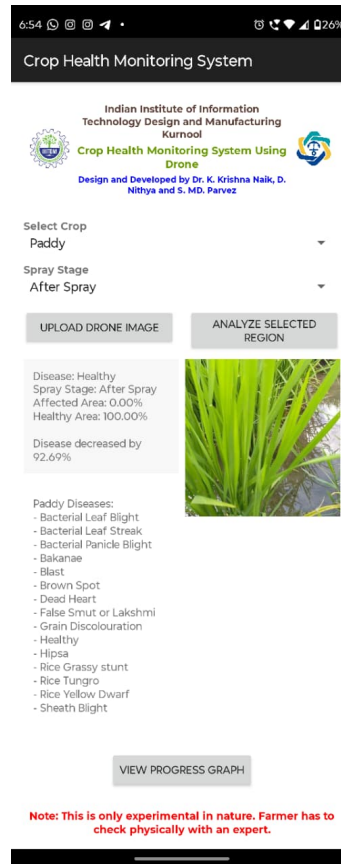


FIGURE 6.34: After Spray Disease Detection

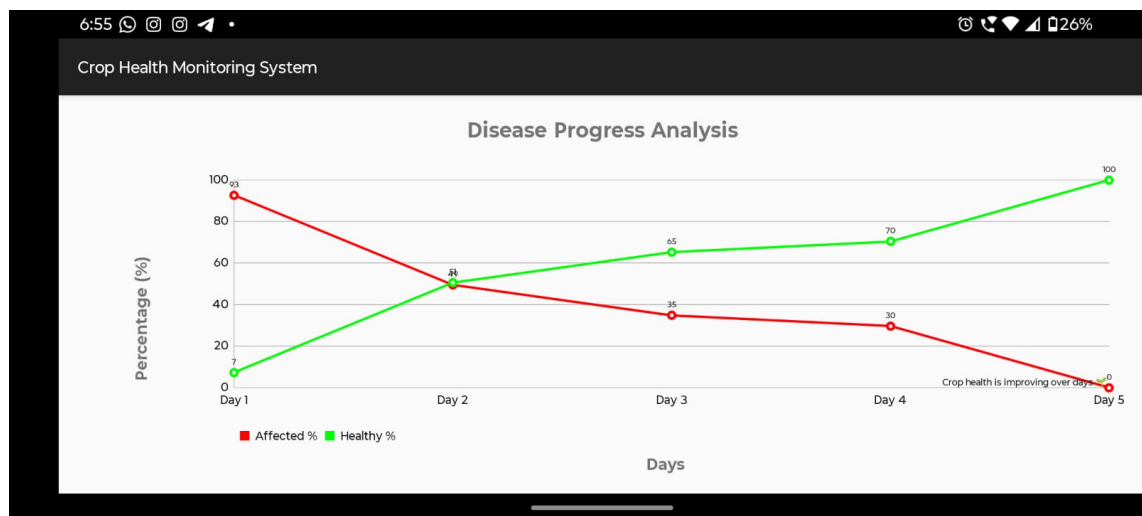
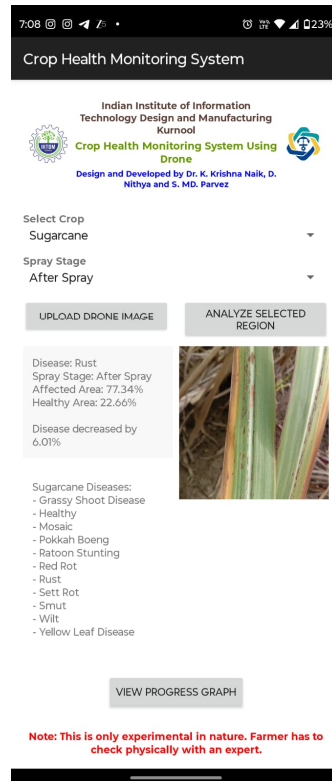
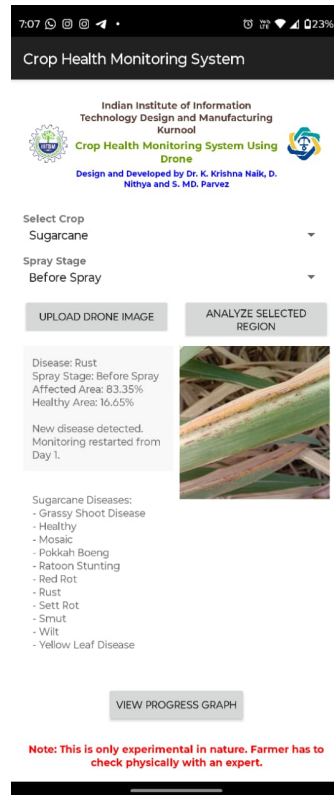


FIGURE 6.35: Disease Progress Analysis

## For Sugarcane Crop



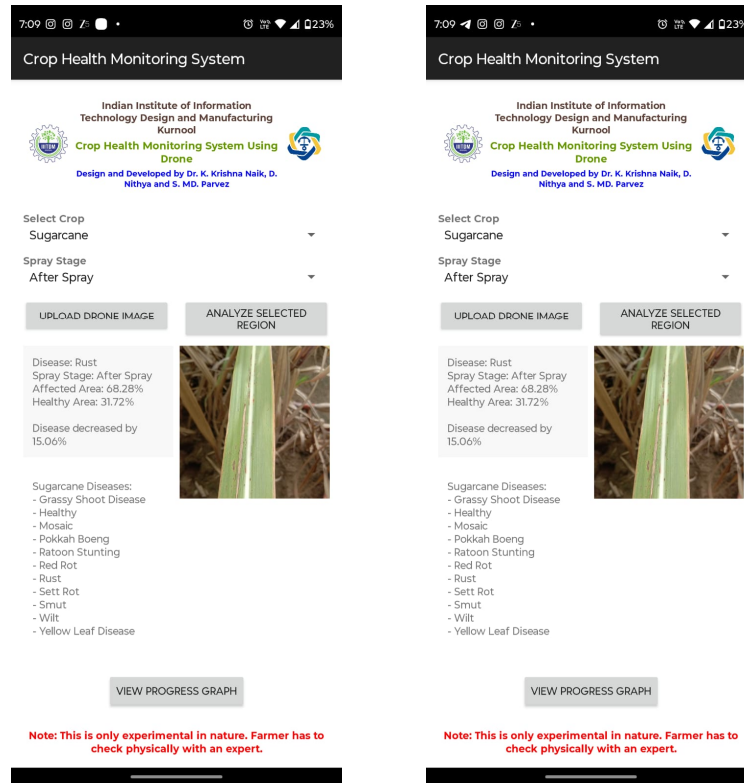


FIGURE 6.36: Before and After Spray Disease Detection

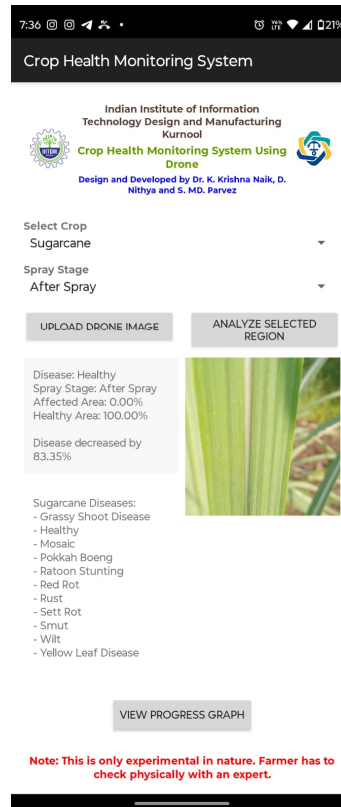


FIGURE 6.37: After Spray Disease Detection

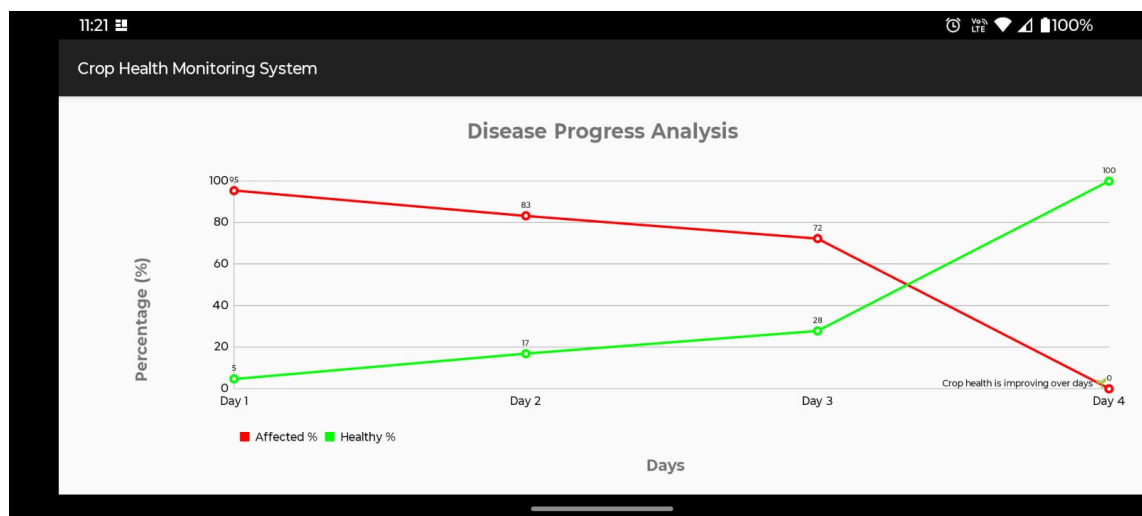
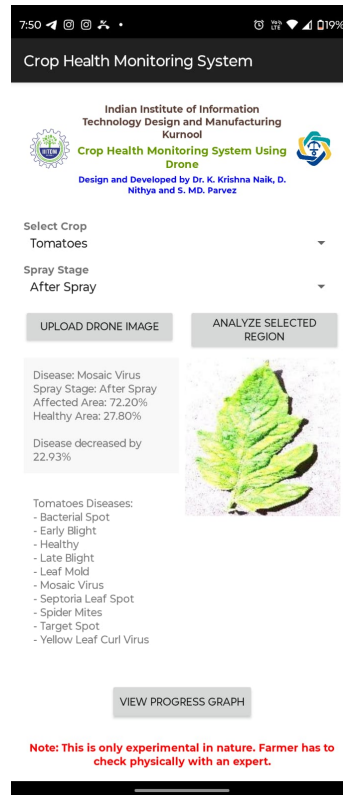
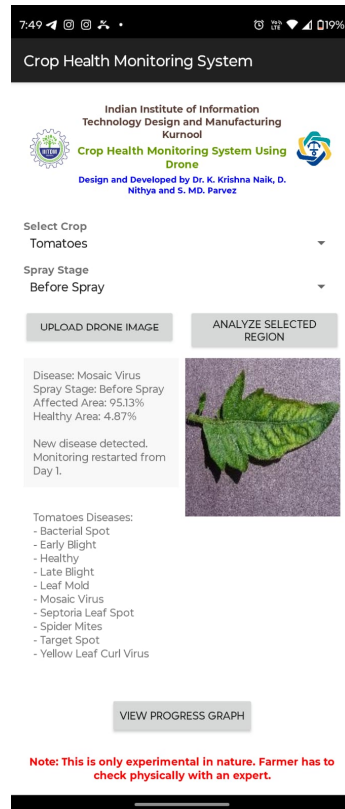


FIGURE 6.38: Disease Progress Analysis

## For Tomato Crop



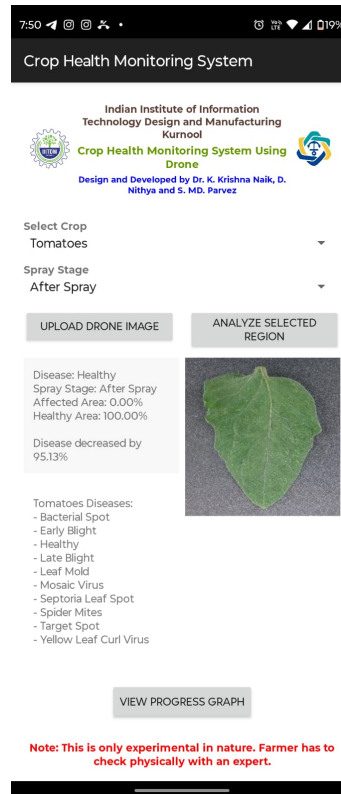


FIGURE 6.39: Before and After Spray Disease Detection

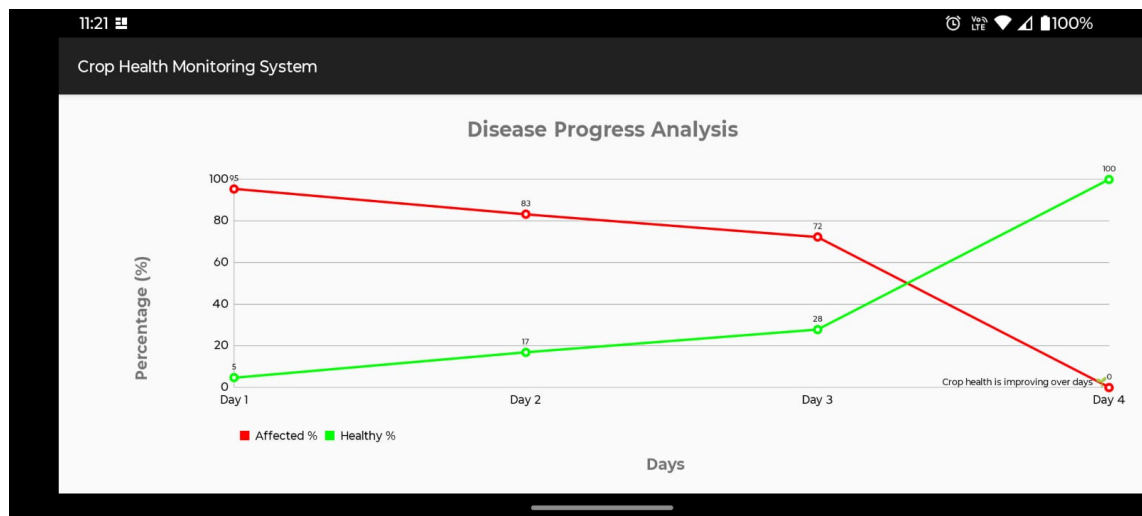


FIGURE 6.40: Disease Progress Analysis

For Wheat Crop

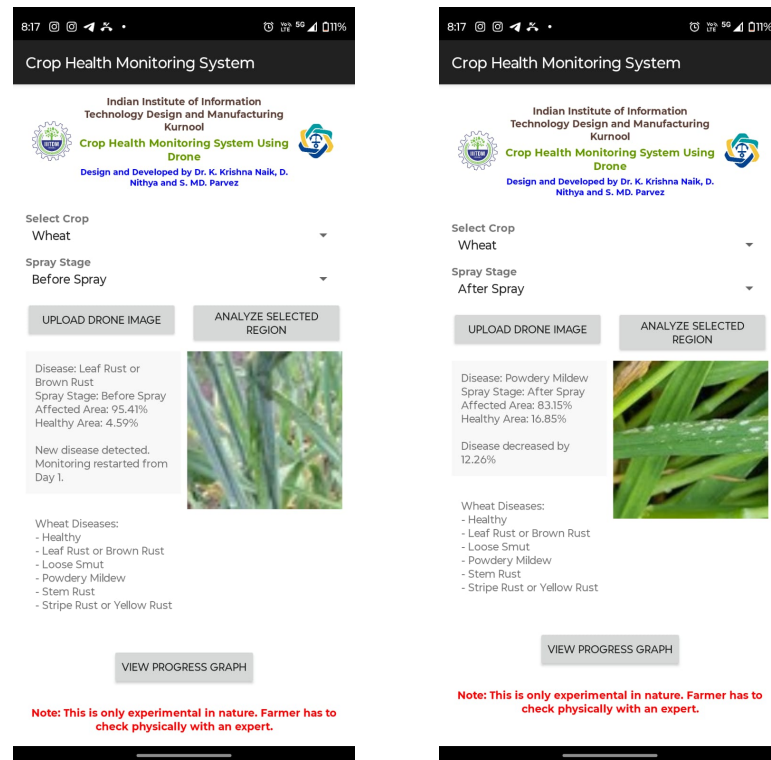


FIGURE 6.41: Before and After Spray Disease Detection

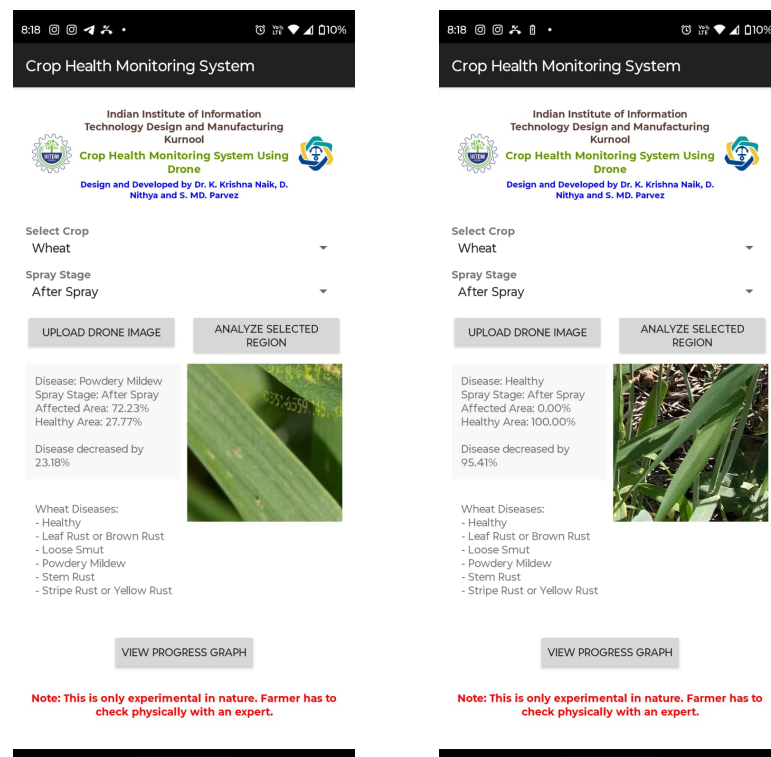


FIGURE 6.42: After Spray Disease Detection

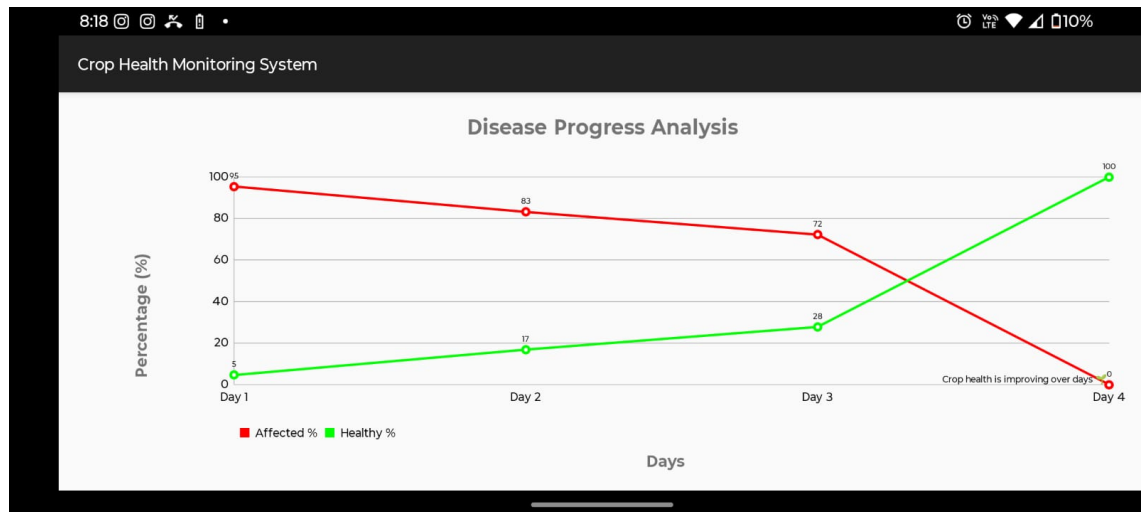


FIGURE 6.43: Disease Progress Analysis

## 6.2 Confusion Matrix Analysis

To better understand how well the trained models perform, confusion matrices were generated for each crop. These matrices provide a clear picture of how accurately the system is able to classify different crop diseases by comparing the actual labels with the predicted ones.

In this project, individual confusion matrices were created for all the selected crops such as Chilli, Coconut, Corn, Cotton, Maize, Onion, Paddy, Sugarcane, Tomatoes, and Wheat. Each matrix helps in visualizing how correctly the model identifies each disease and where it makes mistakes. Most of the values are concentrated along the diagonal of the matrices, which indicates that the majority of the predictions are correct.

There are only a few minor misclassifications observed, mainly between diseases that have very similar visual patterns. This is expected in real-world scenarios where certain crop diseases look almost identical in their early stages. Even then, the model performs consistently well across all crops.

For example, in the maize dataset, diseases such as Cercospora Leaf Spot, Common Rust, and Healthy are identified with high accuracy. Similarly, crops like Tomato and Paddy also show strong classification performance, with only slight confusion between closely related disease categories.

To maintain consistency throughout the system, a predefined mapping between crops and their respective disease classes was used in the application. This ensures that each confusion matrix corresponds exactly to the correct crop model during both training and real-time prediction.

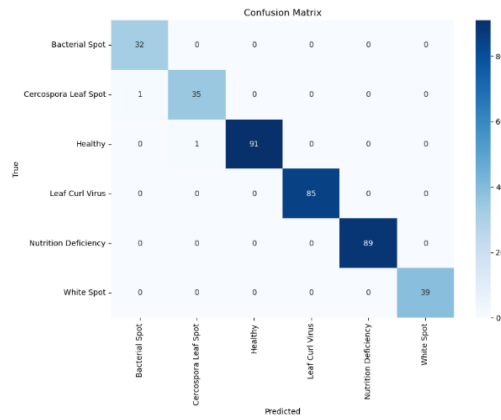


FIGURE 6.44: Chilli Crop

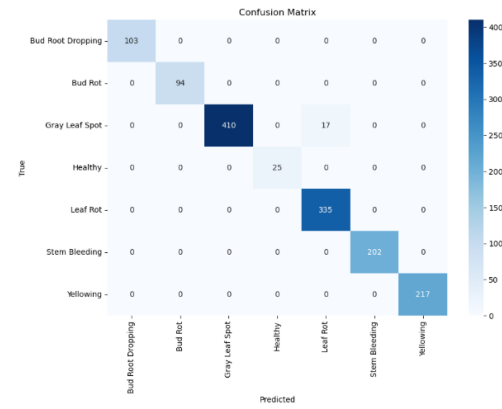


FIGURE 6.45: Coconut Crop

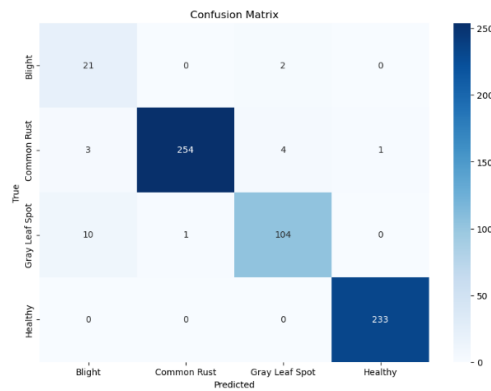


FIGURE 6.46: Corn Crop

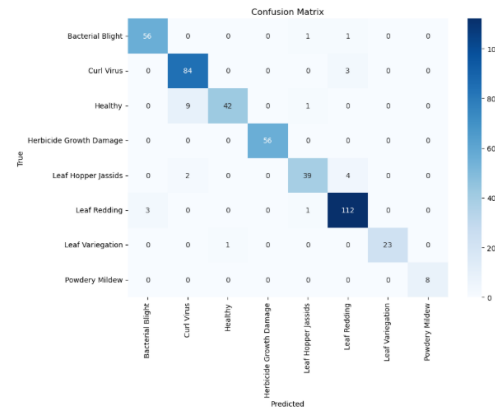


FIGURE 6.47: Cotton Crop

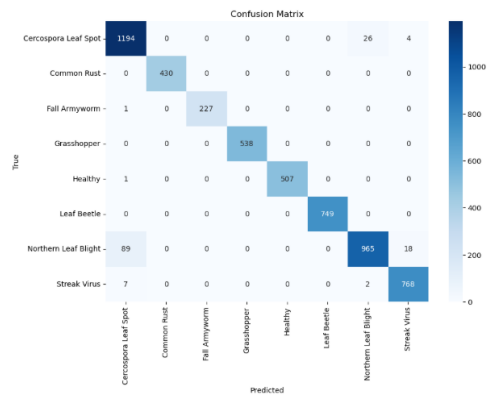


FIGURE 6.48: Maize Crop

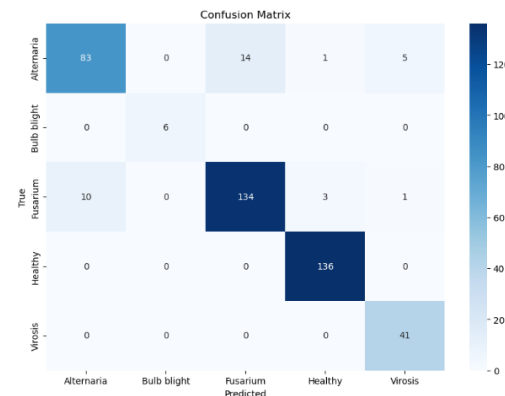


FIGURE 6.49: Onion Crop

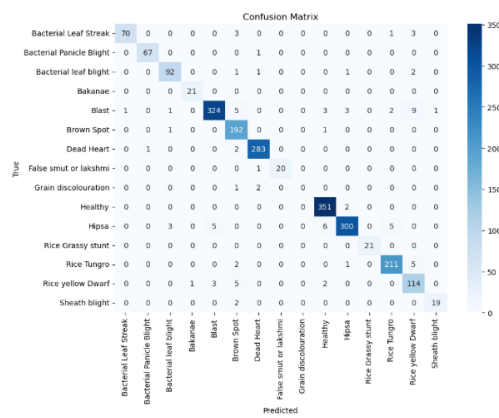


FIGURE 6.50: Paddy Crop

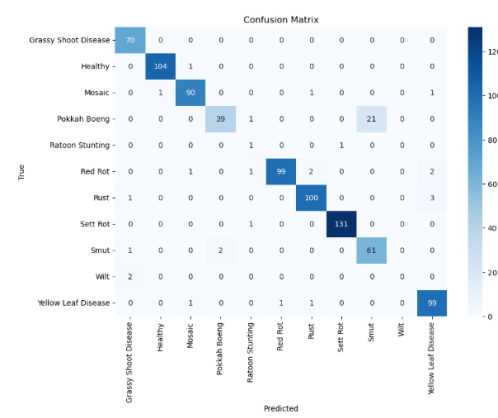


FIGURE 6.51: Sugarcane Crop

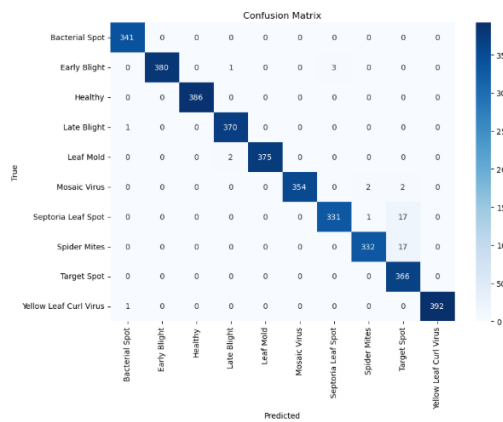


FIGURE 6.52: Tomatos Crop

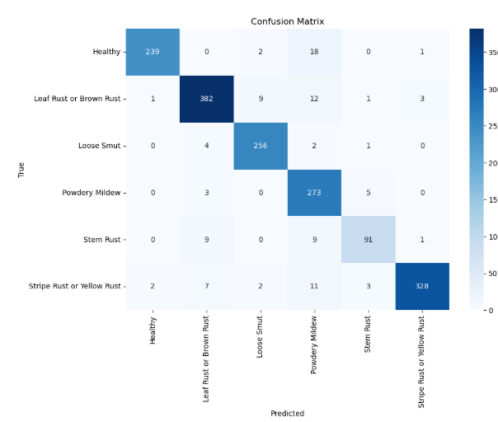


FIGURE 6.53: Wheat Crop

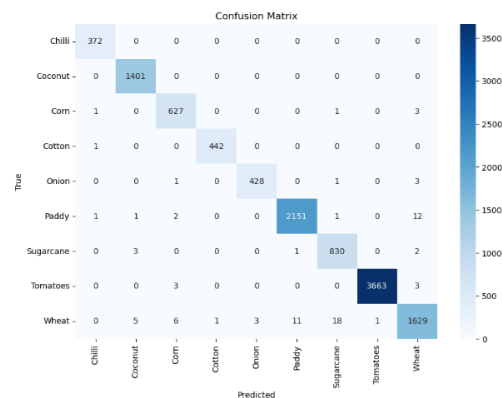


FIGURE 6.54: Crop Verifier

Overall, the confusion matrix analysis confirms that the proposed system is reliable and effective for crop disease classification. The results demonstrate that the model can accurately distinguish between multiple diseases, making it suitable for practical agricultural applications.

### 6.3 Model Accuracy Evaluation

To assess how well the crop disease detection system performs, accuracy was measured for each crop-specific model. These models were trained using deep learning methods and tested on real-world data to see how reliable they are and how well they perform in practical situations.

The results show that the models achieve strong accuracy across different types of crops. Crops like tomato, maize, and chilli tend to have higher accuracy because their disease patterns are more distinct and easier to identify. On the other hand, crops such as paddy and sugarcane show slightly lower accuracy, mainly because their diseases often look similar and are harder to differentiate.

Overall, the results suggest that the system can detect crop diseases quite effectively and is suitable for real-time use in agricultural monitoring.

A detailed breakdown of the accuracy for each crop can be found below

| S. No. | Crop Name     | Model Accuracy (%) |
|--------|---------------|--------------------|
| 1      | Chilli        | 99.46              |
| 2      | Coconut       | 98.79              |
| 3      | Corn          | 96.68              |
| 4      | Cotton        | 94.17              |
| 5      | Maize         | 89.00              |
| 6      | Onion         | 92.17              |
| 7      | Paddy         | 95.91              |
| 8      | Sugarcane     | 94.52              |
| 9      | Tomatoes      | 98.72              |
| 10     | Wheat         | 93.67              |
| 11     | Crop Verifier | 99.27              |

FIGURE 6.55: Accuracy of Crop Disease Detection Models

## Chapter 7

# Conclusion and Future Work

### 7.1 Conclusion

The Crop Health Monitoring System using Drone presents an efficient and intelligent approach to modern agricultural monitoring by integrating Artificial Intelligence, drone imaging, and mobile application technologies. The system is designed to assist farmers and agricultural professionals in detecting crop diseases, estimating their severity, and continuously monitoring crop health with minimal manual effort.

The system operates by capturing crop images using drones or mobile devices, which are then processed through trained deep learning models. A crop verification model ensures that the selected crop matches the uploaded image, thereby improving the reliability of the disease detection process. Once verified, the disease detection model identifies the type of disease, and HSV-based image processing is applied to estimate the severity by calculating affected and healthy regions. These results are then stored and visualized through a user-friendly Android application.

One of the key strengths of this system lies in its ability to provide real-time analysis. The integration of ONNX Runtime within the Android application enables fast and efficient model inference directly on the device, eliminating the need for continuous internet connectivity. Additionally, the graph-based visualization feature allows users to monitor crop health trends over time, making it easier to evaluate the effectiveness of treatments such as pesticide application.

The system was evaluated using multiple datasets and real-time drone images, demonstrating strong performance in both controlled and field environments. The crop

verification model achieved an accuracy of approximately 99%, ensuring correct input validation. The disease detection models achieved accuracies ranging from 89% to 99%, with an overall average accuracy of around 94%. In real-world drone-based testing, the system maintained an accuracy between 88% and 92%, confirming its practical applicability.

The severity estimation module successfully quantified crop health by calculating affected and healthy percentages, providing a more detailed understanding compared to simple visual inspection. Furthermore, the system effectively tracked crop recovery using “Before Spray” and “After Spray” comparisons, enabling users to make informed decisions.

The outcome of the project, the following has been achieved.

1. Developed a crop verification model with 99% accuracy, ensuring reliable input validation.
2. Implemented multiple crop-specific disease detection models achieving an average accuracy of 94%.
3. Achieved real-time prediction performance with inference time between 60 ms to 110 ms per image.
4. Successfully performed field-level testing using drone images with 88%–92% accuracy.
5. Implemented HSV-based severity estimation to quantify affected and healthy crop areas.
6. Developed an Android application for real-time analysis, visualization, and user interaction.
7. Enabled graph-based monitoring to track crop health trends over multiple days.

Overall, the proposed system provides a robust, scalable, and practical solution for precision agriculture. It reduces dependency on manual inspection, improves accuracy in disease detection, and supports timely decision-making, thereby contributing to increased agricultural productivity and sustainability.

## **7.2 Future Work**

Although the proposed system demonstrates strong performance and practical usability, there are several opportunities for further enhancement and expansion.

In future work, the system can be extended to support a wider range of crops and diseases by training models on larger and more diverse datasets. This would improve generalization and make the system applicable across different agricultural regions.

The integration of advanced deep learning models such as Vision Transformers or improved convolutional architectures can further enhance detection accuracy, especially for crops with visually similar disease patterns. Additionally, incorporating multispectral or hyperspectral imaging from drones can provide deeper insights into crop health that are not visible in standard RGB images.

Another important enhancement would be the inclusion of cloud-based storage and analytics. This would allow large-scale data collection, centralized monitoring, and advanced analysis using big data techniques. Farmers could also access historical data and generate detailed reports for long-term planning.

The system can also be expanded into a complete agricultural management platform by integrating features such as:

- Weather data analysis for predicting disease outbreaks
- Soil health monitoring using IoT sensors
- Automated recommendations for pesticide and fertilizer usage

Furthermore, developing multilingual support within the mobile application would make the system more accessible to farmers from different regions. Voice-based interaction can also be introduced to assist users with limited technical knowledge.

Another potential improvement is the integration of automated drone navigation for continuous field monitoring. This would enable periodic data collection without manual intervention, making the system fully autonomous.

Finally, the system can be deployed as a large-scale precision agriculture solution by integrating it with government agricultural programs and smart farming initiatives, thereby benefiting a wider farming community.

# Appendix A

## Application Code

### A.1 Installation Libraries Requirements :

The development and implementation of the Crop Health Monitoring System require several software libraries for image processing, deep learning, model evaluation, and deployment. These libraries ensure efficient model training, validation, and integration into the Android application.

The required libraries are listed below:

1. NumPy – Used for numerical computations and array operations
2. PyTorch (Torch & Torchvision) – Used for training deep learning models such as DenseNet-201
3. ONNX Runtime – Used for running trained models efficiently in mobile applications
4. OpenCV (cv2) – Used for image processing operations
5. Matplotlib – Used for plotting graphs such as confusion matrix
6. Seaborn – Used for visualizing confusion matrix heatmaps
7. Scikit-learn – Used for evaluation metrics such as accuracy, confusion matrix, and classification report
8. Pillow (PIL) – Used for image loading and preprocessing
9. tqdm – Used for displaying training progress

10. OS `shutil` – Used for dataset handling and file operations

These libraries collectively support the complete pipeline from dataset preparation to model deployment.

## A.2 Implementation of the Crop Health Monitoring System

### A.2.1 Crop Verifier Model Training

The crop verifier model is designed to confirm whether an uploaded image actually belongs to a recognized crop category before passing it to the disease classification stage. The training pipeline begins by preparing the dataset through two essential preprocessing steps. First, the dataset is automatically split into training and validation sets using an 80:20 ratio, ensuring that the model is evaluated on data it has never seen during training. Second, a corruption check is performed across all image files to detect and remove any broken or unreadable images that could disrupt the training process. To improve the model's ability to generalize across different field conditions, a set of data augmentation techniques is applied during training. These include random cropping, horizontal flipping, random rotation up to 20 degrees, and color jitter adjustments for brightness, contrast, and saturation. The validation set, on the other hand, is only resized and center-cropped to maintain consistent evaluation conditions.

The model architecture used for crop verification is **DenseNet201**, a densely connected convolutional neural network pre-trained on the ImageNet dataset. To adapt the model for this specific task, the final classifier layer is replaced with a custom fully connected head consisting of a 512-neuron linear layer, a ReLU activation, a dropout layer with a rate of 0.4 to prevent overfitting, and a final output layer with LogSoftmax activation. During training, the earlier feature extraction layers are frozen, and only the last 40 layers are fine-tuned to retain the pre-learned visual representations while adapting to crop-specific features. A confidence threshold of 0.75 is applied during evaluation, meaning that any prediction where the model's confidence falls below this level is rejected and marked as unrecognized. This ensures that only high-confidence predictions are counted, improving the overall reliability of the verifier. After evaluation, a confusion matrix and a detailed classification report are generated to measure performance across all crop classes. Finally, the trained model is exported in **ONNX format** (opset version 12) to enable cross-platform deployment on both the GUI interface and the Android application.

```

1  import os, random, shutil, numpy as np
2  import matplotlib.pyplot as plt
3  from PIL import Image, ImageFile
4  import torch, torch.nn as nn, torch.optim as optim
5  from torchvision import datasets, models, transforms
6  from torch.utils.data import DataLoader
7  from sklearn.metrics import confusion_matrix, classification_report
8  import seaborn as sns
9
10 ImageFile.LOAD_TRUNCATED_IMAGES = True
11 torch.backends.cudnn.benchmark = True
12
13 def safe_pil_loader(path):
14     try:
15         with open(path, "rb") as f:
16             img = Image.open(f)
17             return img.convert("RGB")
18     except Exception:
19         return Image.new("RGB", (224, 224), (0, 0, 0))
20
21 def remove_corrupted_images(root_dir):
22     removed = 0
23     for subdir, _, files in os.walk(root_dir):
24         for f in files:
25             if not f.lower().endswith((".jpg", ".jpeg", ".png")):
26                 continue
27             path = os.path.join(subdir, f)
28             try:
29                 img = Image.open(path); img.verify()
30                 img = Image.open(path); img.convert("RGB")
31             except Exception:
32                 try: os.remove(path); removed += 1
33                 except: pass
34
35 def split_dataset(root, train_ratio=0.8):
36     train_dir = os.path.join(root, "train")
37     val_dir = os.path.join(root, "val")
38     if os.path.exists(train_dir) and os.path.exists(val_dir):
39         return
40     os.makedirs(train_dir, exist_ok=True)
41     os.makedirs(val_dir, exist_ok=True)
42     for cls in os.listdir(root):
43         cls_path = os.path.join(root, cls)

```

```

44         if not os.path.isdir(cls_path) or cls in ["train", "val"]:
45             continue
46         images = [f for f in os.listdir(cls_path)
47                     if f.lower().endswith((".jpg", ".jpeg", ".png"))]
48         if len(images) < 5: continue
49         random.shuffle(images)
50         split = int(len(images) * train_ratio)
51         for img in images[:split]:
52             os.makedirs(os.path.join(train_dir, cls), exist_ok=True)
53             shutil.copy(os.path.join(cls_path, img),
54                         os.path.join(train_dir, cls, img))
55         for img in images[split:]:
56             os.makedirs(os.path.join(val_dir, cls), exist_ok=True)
57             shutil.copy(os.path.join(cls_path, img),
58                         os.path.join(val_dir, cls, img))
59
60 def TrainCropVerifier_FULL():
61     dataset_root = r"C:\Users\USER\Documents\MATLAB\Crop Verifiers"
62     output_dir    = r"C:\Users\USER\Documents\MATLAB"
63     ckpt_dir      = os.path.join(output_dir, "VerifiersCheckpoints")
64     os.makedirs(ckpt_dir, exist_ok=True)
65     confidence_threshold = 0.75
66     onnx_path = os.path.join(output_dir, "crop_verifier.onnx")
67
68     split_dataset(dataset_root)
69     remove_corrupted_images(dataset_root)
70
71     transforms_dict = {
72         "train": transforms.Compose([
73             transforms.Resize((256, 256)),
74             transforms.RandomCrop(224),
75             transforms.RandomHorizontalFlip(),
76             transforms.RandomRotation(20),
77             transforms.ColorJitter(0.2, 0.2, 0.2),
78             transforms.ToTensor(),
79             transforms.Normalize([0.485, 0.456, 0.406],
80                                  [0.229, 0.224, 0.225]))),
81         "val": transforms.Compose([
82             transforms.Resize((256, 256)),
83             transforms.CenterCrop(224),
84             transforms.ToTensor(),
85             transforms.Normalize([0.485, 0.456, 0.406],
86                                  [0.229, 0.224, 0.225]))),
87     }
88     datasets_dict = {

```

```

89         x: datasets.ImageFolder(
90             os.path.join(dataset_root, x),
91             transform=transforms_dict[x],
92             loader=safe_pil_loader)
93     for x in ["train", "val"]
94 }
95 loaders = {
96     x: DataLoader(datasets_dict[x], batch_size=32,
97                   shuffle=False, num_workers=0)
98     for x in ["train", "val"]
99 }
100 classes      = datasets_dict["train"].classes
101 num_classes   = len(classes)
102 device        = torch.device(
103     "cuda" if torch.cuda.is_available() else "cpu")
104
105 model = models.densenet201(weights="IMAGENET1K_V1")
106 for p in model.features.parameters():
107     p.requires_grad = False
108 for p in list(model.features.parameters())[-40:]:
109     p.requires_grad = True
110 model.classifier = nn.Sequential(
111     nn.Linear(model.classifier.in_features, 512),
112     nn.ReLU(), nn.Dropout(0.4),
113     nn.Linear(512, num_classes),
114     nn.LogSoftmax(dim=1))
115 model.to(device)
116
117 best_model_path = os.path.join(ckpt_dir, "best_model.pth")
118 model.load_state_dict(
119     torch.load(best_model_path, map_location=device))
120 model.eval()
121
122 y_true, y_pred = [], []
123 with torch.no_grad():
124     for x, y in loaders["val"]:
125         x = x.to(device)
126         out = model(x)
127         probs = torch.exp(out)
128         conf, preds = torch.max(probs, 1)
129         preds = torch.where(
130             conf >= confidence_threshold, preds,
131             torch.tensor(-1).to(device))
132         y_true.extend(y.numpy())
133         y_pred.extend(preds.cpu().numpy())

```

```

134
135     filtered_true = [t for t,p in zip(y_true,y_pred) if p != -1]
136     filtered_pred = [p for t,p in zip(y_true,y_pred) if p != -1]
137     print(classification_report(filtered_true, filtered_pred,
138                               target_names=classes,
139                               zero_division=0))
140
141     torch.onnx.export(
142         model, torch.randn(1,3,224,224,device=device),
143         onnx_path,
144         input_names=["input"], output_names=["output"],
145         opset_version=12)
146
147 if __name__ == "__main__":
148     TrainCropVerifier_FULL()

```

LISTING A.1: Crop Verifier Model Training

### A.2.2 Individual Crop Disease Classification Model Training

Each of the seven selected crops is trained as an independent disease classification model using the same DenseNet201 backbone. The training script is designed to be interactive, allowing the user to select a specific crop from the available dataset folders before training begins. This modular approach ensures that each crop model is optimized for its own set of disease categories without interference from other crops.

The training configuration uses a batch size of 16, a learning rate of  $3 \times 10^{-4}$ , and runs for a maximum of 60 epochs. To prevent unnecessary training when the model has already converged, an early stopping mechanism is implemented with a patience of 5 epochs, meaning training is halted if no improvement in validation accuracy is observed after five consecutive epochs. A learning rate scheduler is also employed, which reduces the learning rate by a factor of 0.6 when validation accuracy plateaus, helping the model fine-tune itself more precisely in the later stages of training.

Model checkpoints are saved after every epoch, and the best-performing model based on validation accuracy is separately preserved as `best_model.pth`. If a checkpoint already exists from a previous session, the training automatically resumes from where it left off, avoiding redundant retraining. Once training is complete, the best model is exported in ONNX format for deployment on the mobile application and GUI interface.

```

1 import os, shutil, random
2 import matplotlib.pyplot as plt
3 import torch, torch.nn as nn, torch.optim as optim
4 from torch.utils.data import DataLoader
5 from torchvision import datasets, models, transforms
6 from PIL import Image, ImageFile
7 from tqdm import tqdm
8 ImageFile.LOAD_TRUNCATED_IMAGES = True
9
10 def create_train_val_split(data_path, train_ratio=0.8):
11     train_dir = os.path.join(data_path, "train")
12     val_dir = os.path.join(data_path, "val")
13     if os.path.exists(train_dir) and os.path.exists(val_dir):
14         return
15     os.makedirs(train_dir, exist_ok=True)
16     os.makedirs(val_dir, exist_ok=True)
17     for cls in os.listdir(data_path):
18         cls_path = os.path.join(data_path, cls)
19         if not os.path.isdir(cls_path) or cls in ["train", "val"]:
20             continue
21         imgs = [f for f in os.listdir(cls_path)
22                 if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
23         random.shuffle(imgs)
24         split = int(len(imgs) * train_ratio)
25         for d, subset in [(train_dir, imgs[:split]),
26                           (val_dir, imgs[split:]):
27             os.makedirs(os.path.join(d, cls), exist_ok=True)
28             for img in subset:
29                 shutil.copy(os.path.join(cls_path, img),
30                             os.path.join(d, cls, img))
31
32 def TrainSingleCrop():
33     total_epochs = 60
34     patience = 5
35     batch_size = 16
36     learning_rate = 3e-4
37
38     base_data_path = r"C:\Users\USER\Documents\MATLAB\M Tech Project\
39 Latest"
40     checkpoint_base = r"C:\Users\USER\Documents\MATLAB\M Tech Project\
41 Checkpoints"
42     models_folder = r"C:\Users\USER\Documents\MATLAB\M Tech Project\
43 Models"
44     os.makedirs(models_folder, exist_ok=True)

```

```

42
43     device = torch.device("cuda" if torch.cuda.is_available()
44                           else "cpu")
45     crops = [f for f in os.listdir(base_data_path)
46              if os.path.isdir(os.path.join(base_data_path, f))]
47     choice = input("Enter crop number to train: ")
48     crop = crops[int(choice)-1]
49
50     data_path = os.path.join(base_data_path, crop)
51     checkpoint_dir = os.path.join(checkpoint_base, crop)
52     os.makedirs(checkpoint_dir, exist_ok=True)
53     onnx_path = os.path.join(models_folder, f"trained_{crop}.onnx")
54
55     create_train_val_split(data_path)
56
57     data_transforms = {
58         "train": transforms.Compose([
59             transforms.Resize((256, 256)),
60             transforms.RandomCrop(224),
61             transforms.RandomHorizontalFlip(),
62             transforms.RandomRotation(20),
63             transforms.ColorJitter(0.2, 0.2, 0.2),
64             transforms.ToTensor(),
65             transforms.Normalize([0.485, 0.456, 0.406],
66                                 [0.229, 0.224, 0.225]))],
67         "val": transforms.Compose([
68             transforms.Resize((256, 256)),
69             transforms.CenterCrop(224),
70             transforms.ToTensor(),
71             transforms.Normalize([0.485, 0.456, 0.406],
72                                 [0.229, 0.224, 0.225]))])
73     }
74     dataset = {
75         x: datasets.ImageFolder(
76             os.path.join(data_path, x), data_transforms[x])
77         for x in ["train", "val"]
78     }
79     dataloaders = {
80         x: DataLoader(dataset[x], batch_size=batch_size,
81                       shuffle=True, num_workers=0)
82         for x in ["train", "val"]
83     }
84     num_classes = len(dataset["train"].classes)
85
86     model = models.densenet201(weights="IMAGENET1K_V1")

```

```

87     model.classifier = nn.Sequential(
88         nn.Linear(model.classifier.in_features, 512),
89         nn.ReLU(), nn.Dropout(0.4),
90         nn.Linear(512, num_classes),
91         nn.LogSoftmax(dim=1))
92     model.to(device)
93
94     criterion = nn.NLLLoss()
95     optimizer = optim.Adam(model.parameters(), lr=learning_rate)
96     scheduler = optim.lr_scheduler.ReduceLROnPlateau(
97         optimizer, mode='max', patience=3, factor=0.6)
98
99     checkpoint_path = os.path.join(checkpoint_dir, "checkpoint.pth")
100    best_model_path = os.path.join(checkpoint_dir, "best_model.pth")
101    start_epoch, best_acc, patience_counter = 0, 0.0, 0
102
103    if os.path.exists(best_model_path):
104        model.load_state_dict(
105            torch.load(best_model_path, map_location=device))
106    if os.path.exists(checkpoint_path):
107        ckpt = torch.load(checkpoint_path, map_location=device)
108        if "model_state_dict" in ckpt:
109            optimizer.load_state_dict(ckpt["optimizer_state_dict"])
110            start_epoch = ckpt["epoch"] + 1
111
112    for epoch in range(start_epoch, total_epochs):
113        for phase in ["train", "val"]:
114            model.train() if phase=="train" else model.eval()
115            running_loss, correct = 0, 0
116            for inputs, labels in tqdm(dataloaders[phase]):
117                inputs, labels = inputs.to(device), labels.to(device)
118                optimizer.zero_grad()
119                with torch.set_grad_enabled(phase=="train"):
120                    outputs = model(inputs)
121                    _, preds = torch.max(outputs, 1)
122                    loss = criterion(outputs, labels)
123                    if phase=="train":
124                        loss.backward(); optimizer.step()
125                    running_loss += loss.item() * inputs.size(0)
126                    correct      += torch.sum(preds == labels)
127            epoch_acc = correct.double() / len(dataset[phase])
128            if phase=="val":
129                scheduler.step(epoch_acc)
130                if epoch_acc > best_acc:
131                    best_acc = epoch_acc; patience_counter = 0

```

```

132         torch.save(model.state_dict(), best_model_path)
133     else:
134         patience_counter += 1
135         torch.save({"epoch": epoch,
136                   "model_state_dict": model.state_dict(),
137                   "optimizer_state_dict":
138                       optimizer.state_dict()},
139                   checkpoint_path)
140         if patience_counter >= patience: break
141
142     torch.onnx.export(
143         model, torch.randn(1,3,224,224,device=device), onnx_path,
144         input_names=["input"], output_names=["output"],
145         opset_version=12)
146     print(f"Best Accuracy: {best_acc*100:.2f}%")
147
148 if __name__ == "__main__":
149     TrainSingleCrop()

```

LISTING A.2: Individual Crop Disease Classification Model Training

### A.2.3 Confusion Matrix and Classification Report Generation

Once each crop model is trained and saved in ONNX format, a separate evaluation script is used to measure its performance on the training dataset. This script loads the exported ONNX model and runs it through ONNX Runtime, which allows the model to be evaluated independently of the PyTorch framework. Before inference begins, all images in the dataset are validated using a custom corruption check function, ensuring that no broken files disrupt the evaluation process.

Each image is passed individually through the ONNX model after applying the standard preprocessing pipeline — resizing, center cropping, tensor conversion, and normalization. The predicted class label is determined by selecting the index with the highest output score using an argmax operation. After processing the entire dataset, the overall model accuracy is computed, and a **confusion matrix** is plotted as a heatmap using Seaborn to visually illustrate how well the model distinguishes between different disease classes. Additionally, a **classification report** is printed, providing detailed per-class metrics including precision, recall, and F1-score, which together offer a comprehensive picture of the model's strengths and weaknesses across all disease categories.

```

1 import onnxruntime as ort
2 import numpy as np, torch
3 from torchvision import datasets, transforms
4 from sklearn.metrics import (confusion_matrix, classification_report,
5                               accuracy_score)
6 import matplotlib.pyplot as plt, seaborn as sns
7 from PIL import Image, ImageFile
8 ImageFile.LOAD_TRUNCATED_IMAGES = True
9
10 def is_valid_image(path):
11     try:
12         with Image.open(path) as img: img.verify()
13         with Image.open(path) as img: img.load()
14         return True
15     except Exception:
16         return False
17
18 onnx_model_path = (r"C:\Users\USER\Documents\MATLAB\M Tech Project"
19                   r"\Models\trained_Maize.onnx")
20 session = ort.InferenceSession(onnx_model_path,
21                                providers=['CPUExecutionProvider'])
22 input_name = session.get_inputs()[0].name
23 output_name = session.get_outputs()[0].name
24
25 test_transform = transforms.Compose([
26     transforms.Resize(256), transforms.CenterCrop(224),
27     transforms.ToTensor(),
28     transforms.Normalize([0.485, 0.456, 0.406],
29                           [0.229, 0.224, 0.225])])
30
31 test_dir = (r"C:\Users\USER\Documents\MATLAB\M Tech Project"
32            r"\Latest\Maize\train")
33 dataset = datasets.ImageFolder(test_dir,
34                                transform=test_transform,
35                                is_valid_file=is_valid_image)
36 dataloader = torch.utils.data.DataLoader(
37     dataset, batch_size=16, shuffle=False)
38 class_names = dataset.classes
39
40 all_preds, all_labels = [], []
41 for images, labels in dataloader:
42     images_np = images.numpy()
43     for i in range(images_np.shape[0]):
44         img = np.expand_dims(images_np[i], axis=0)

```

```
45         result = session.run([output_name],
46                               {input_name: img.astype(np.float32)})
47         all_preds.append(np.argmax(result[0]))
48         all_labels.extend(labels.numpy())
49
50 accuracy = accuracy_score(all_labels, all_preds)
51 print(f"Model Accuracy: {accuracy*100:.2f}%")
52
53 cm = confusion_matrix(all_labels, all_preds)
54 plt.figure(figsize=(10,7))
55 sns.heatmap(cm, annot=True, fmt="d",
56             xticklabels=class_names,
57             yticklabels=class_names, cmap='Blues')
58 plt.xlabel("Predicted"); plt.ylabel("True")
59 plt.title("Confusion Matrix"); plt.show()
60
61 print(classification_report(all_labels, all_preds,
62                             target_names=class_names))
```

LISTING A.3: Confusion Matrix and Classification Report

# Bibliography

- [1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [2] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.
- [3] P. Singh, V. Sharma, and A. Jain, “Plant disease detection using deep learning and convolutional neural networks,” *International Journal of Computer Applications*, vol. 180, no. 47, pp. 5–9, 2018.
- [4] S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, “Deep neural networks based recognition of plant diseases by leaf image classification,” *Computational Intelligence and Neuroscience*, vol. 2016, pp. 1–11, 2016.
- [5] J. Too, L. Yujian, S. Njuki, and L. Yingchun, “A comparative study of fine-tuning deep learning models for plant disease identification,” *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.
- [6] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.
- [7] M. Sharada, R. S. Rajpurohit, and P. Yadav, “Drone based crop health monitoring system using image processing,” *International Journal of Engineering Research & Technology*, vol. 9, no. 7, pp. 555–560, 2020.
- [8] D. H. Pham, T. T. Do, and H. H. Pham, “Aerial imagery-based rice disease detection using deep learning,” *IEEE Access*, vol. 8, pp. 203 859–203 871, 2020.
- [9] J. G. A. Barbedo, “Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification,” *Computers and Electronics in Agriculture*, vol. 153, pp. 46–53, 2018.

- 
- [10] H. P. Ramcharan, T. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. Hughes, “Deep learning for image-based cassava disease detection,” *Frontiers in Plant Science*, vol. 8, p. 1852, 2017.
  - [11] M. Brahimi, K. Boukhalfa, and A. Moussaoui, “Deep learning for tomato diseases: classification and symptoms visualization,” *Applied Artificial Intelligence*, vol. 31, no. 4, pp. 299–315, 2017.
  - [12] S. Ramesh, S. Vydeki, and K. S. Reddy, “Plant disease detection using machine learning algorithms,” *Procedia Computer Science*, vol. 167, pp. 1808–1815, 2020.
  - [13] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
  - [14] X. Zhang, Y. Qiao, F. Meng, C. Fan, and M. Zhang, “Identification of maize leaf diseases using improved deep convolutional neural networks,” *IEEE Access*, vol. 6, pp. 30 370–30 377, 2018.
  - [15] R. Zhang, Y. Guo, and Y. Chen, “Crop disease recognition based on deep convolutional neural networks,” *Computers and Electronics in Agriculture*, vol. 170, p. 105238, 2020.